Proyectos de Programación
Kenken

Versión 3.0

Subgrupo 43.3
Álex Jiménez Sanso - alex.jimenez.sanso
Carlos Lesmes Leo - carlos.lesmes
Federica Caretta - federica.caretta
Jan Torrent Soler - jan.torrent

Junio 2024

Índice general

1. Relación de las clases implementadas por cada miembro del equipo **1**

2. Casos de Uso **4**

2.1. Crear Perfil 5
2.2. Log In 5
2.3. Log Out 6
2.4. Eliminar Perfil 6
2.5. Importar Kenken 6
2.6. Generar Kenken aleatorio 7
2.7. Generar Kenken personalizado 7
2.8. Consultar Ranking 7
2.9. Consultar Estadísticas 8
2.10. Consultar Normas 8
2.11. Configurar Ajustes 9
2.12. Salir 9
2.13. Jugar Partida 9
2.14. Rellenar Casilla 9
2.15. Borrar Casilla 10
2.16. Poner Nota 10
2.17. Pedir Hint 10
2.18. Pausar Partida 11
2.19. Reanudar Partida 11
2.20. Cerrar Partida 11
2.21. Reiniciar Partida 11
2.22. Continuar Partida 12

3. Diagrama de clases **13**

3.1. Diagrama de la capa de dominio 13
3.2. Descripción del diagrama de la capa de dominio 14
3.3. Diagrama de la capa de persistencia 30

3.4. Descripción del diagrama de la capa de persistencia 30
3.5. Diagrama de la capa de presentación 32
3.6. Descripción del diagrama de la capa de presentación 32

4. Estructuras de datos y algoritmos **36**
4.1. Resolución de Kenkens 36
4.1.1. Verificador 36
4.1.2. Traductor 36
4.1.3. Solver 36
4.1.4. Kenken 38
4.1.5. Observaciones 38
4.2. Generación de Kenkens 40
4.2.1. Generar solución 40
4.2.2. Definir regiones 43
4.2.3. Comprobar la unicidad de la solución 45
4.2.4. Observaciones 46
4.3. Estructuras de datos a comentar 48

5. Operaciones añadidas **51**
5.1. Media aritmética (AM) 51
5.2. Máximo (max) 53

6. Juegos de prueba **54**
6.1. Juegos de prueba con persistencia 54
6.2. Juegos de prueba sin persistencia 60

Anexos **65**

A. Verificador, en detalle **66**

Capítulo 1

Relación de las clases implementadas por cada miembro del equipo

Álex Jiménez Sanso:

- `Generator.java`
- `GeneratorTest.java`
- `GOperation.java`
- `GOperationTest.java`
- `GSolver.java`
- `GSolverTest.java`
- `Operation.java`
- `OperationTest.java`
- `Identity.java`
- `Addition.java`
- `Subtraction.java`
- `Product.java`
- `Division.java`
- `ArithmeticMean.java`
- `Max.java`
- `Solver.java`

- User.java
- UserTest.java

Carlos Lesmes Leo:

- CtrlLogic.java
- CtrlData.java
- Reader.java
- Writer.java
- DriverCtrlLogic.java
- KenkenAlreadyInPersistence.java
- KenkenNotGenerated.java

Federica Caretta:

- Coordinates.java
- CoordinatesTest.java
- Region.java
- RegionTest.java
- Kenken.java
- KenkenTest.java
- Statistics.java
- StatisticsTest.java
- Game.java
- GameTest.java
- CustomException.java
- GameAlreadyExists.java
- IllegalCellSelection.java
- IllegalGameAction.java
- IllegalMatrixPosition.java
- IllegalScore.java
- CtrlPresentation.java

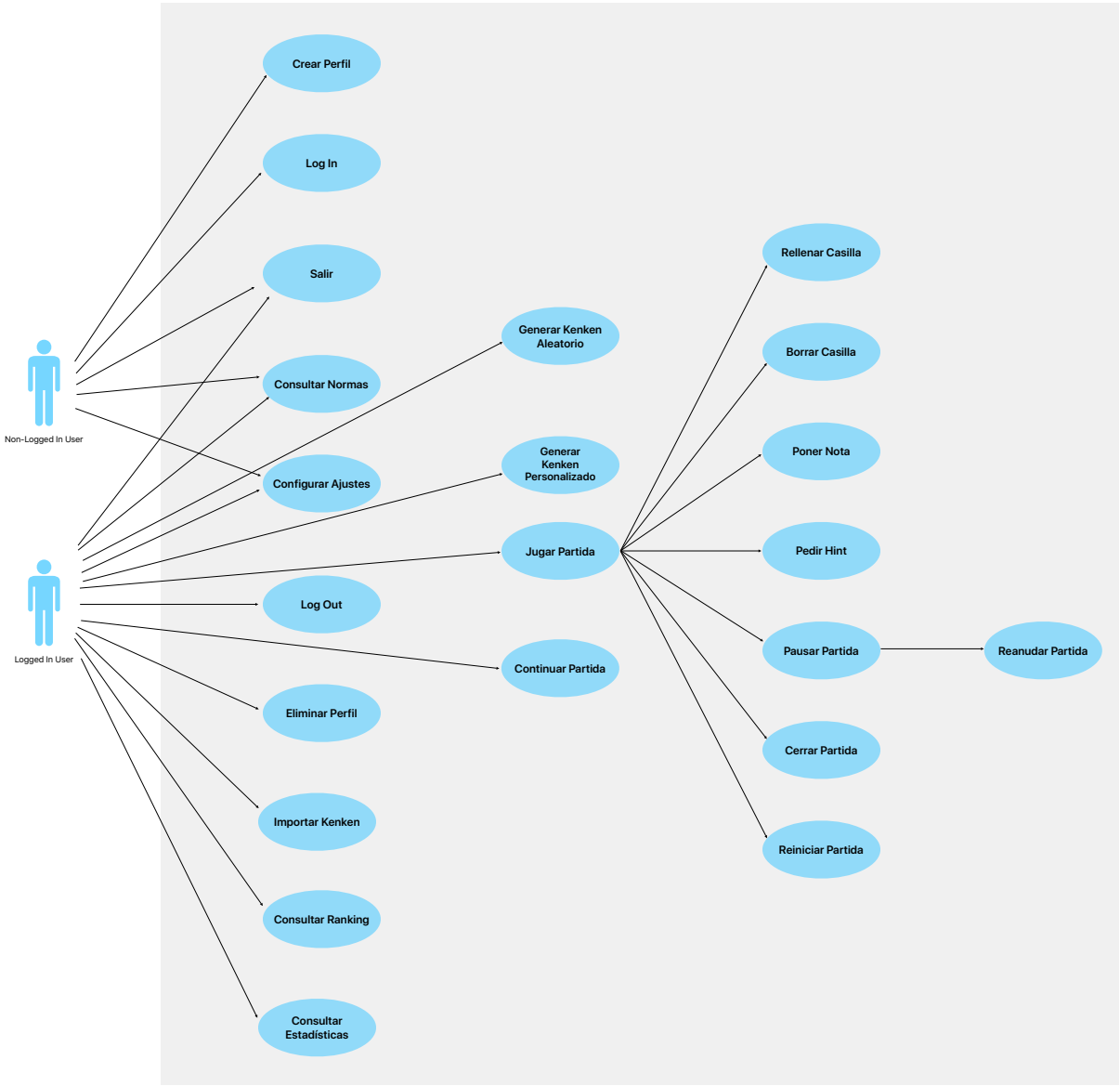
- `MainView.java`
- `GenerationView.java`
- `KenkenDisplayPanel.java`
- `MusicPlayer.java`
- `AppMain.java`

Jan Torrent Soler:

- `Verifier.java`
- `VerifierTest.java`
- `Translator.java`
- `TranslatorTest.java`
- `Ranking.java`
- `RankingTest.java`
- `GameView.java`
- `MainView.java`
- `CtrPresetation.java`
- `AppMain.java`

Capítulo 2

Casos de Uso



2.1. Crear Perfil

Autor: Usuario no registrado
Comportamiento:

1. El usuario introduce el nombre de usuario y la contraseña dos veces (para confirmar).
2. El sistema guarda la información del usuario en un perfil nuevo.
3. El sistema redirige al caso de uso *Log In*.

Errores posibles y usos alternativos:

1.1 Ya existe el nombre de usuario: el sistema pide un nuevo nombre.

1.2 La contraseña y la confirmación de la contraseña no coinciden: el sistema solicita al usuario que introduzca ambos campos.

1.3 Se intenta validar la acción con alguno de los tres campos sin rellenar: el sistema solicita que se rellenen los campos vacíos.

1.4 Si el nombre de usuario contiene un o más espacios, el sistema solicita que se reintroduzca el campo.

1.5 La contraseña no se adecua a los que requisitos siguientes:

- Tener como mínimo 8 caracteres.
- Contiene un carácter especial como mínimo.
- Contiene por lo menos un número.
- Contiene 1 mayúscula.

El sistema solicita que se reintroduzca los campos de contraseña.

2.2. Log In

Autor: Usuario no registrado
Comportamiento:

1. El usuario introduce sus credenciales (una vez el nombre de usuario y una vez la contraseña).
2. El sistema carga la sesión asociada a las credenciales introducidas.

Errores posibles y usos alternativos:

1.1 El nombre de usuario no coincide con ninguno de los nombres de la base de datos: el sistema pide que se reinserte el nombre de usuario.

1.2 La contraseña no coincide con la asociada al nombre de usuario introducido: el sistema pide que se reinserte la contraseña.

2.3. Log Out

Autor: Usuario
Comportamiento:

1. El sistema pide la confirmación de la acción al usuario.
2. El sistema guarda y descarga el perfil del usuario.
3. El sistema da la opción al usuario de iniciar otra sesión.

Errores posibles y usos alternativos:

Nota "Usuario"se refiere a un usuario que haya hecho el log-in.

2.4. Eliminar Perfil

Autor: Usuario
Comportamiento:

1. El sistema pide la confirmación de la acción al usuario.
2. El sistema elimina el perfil del usuario.

Errores posibles y usos alternativos:

1.1 El usuario retira la acción.

2.5. Importar Kenken

Autor: Usuario
Comportamiento:

1. El usuario selecciona el archivo `.txt` que representa el KenKen.
2. El sistema añade el KenKen a aquellos disponibles.

Errores posibles y usos alternativos:

1.1 Si el formato del Kenken no es válido, el sistema informa del error y descarta el Kenken.

1.2 Si el Kenken no tiene solución o tiene solución múltiple, el sistema informa del error y descarta el Kenken.

2.6. Generar Kenken aleatorio

Autor: Usuario
Comportamiento:

1. El usuario selecciona:
   - Las operaciones (entre las 6 disponibles)
   - El tamaño N del tablero.
2. El sistema genera un KenKen válido y lo añade a los disponibles.

Errores posibles y usos alternativos:

1. No se pueden generar Kenkens de tamaño mayor o igual a 100 x 100.

2.7. Generar Kenken personalizado

Autor: Usuario
Comportamiento:

1. El usuario selecciona:
   - El tamaño N del tablero
   - La topología del tablero

Errores posibles y usos alternativos:

1.1 Si las limitaciones fijadas por el usuario no dan lugar a un KenKen válido, el sistema informa del error. No se pueden generar Kenkens de tamaño mayor o igual a 100 x 100.

2.8. Consultar Ranking

Autor: Usuario
Comportamiento:

1. El usuario visualiza una tabla con las 50 primeras posiciones ordenadas por nuestra métrica de experiencia resolviendo Kenken's, la cual se obtiene al resolver Kenken's

según diversos parámetros como el tiempo que le toma al jugador acabar la partida o las dimensiones del Kenken que ha completado. En caso de empate se mantendrá el orden lexicográfico y en caso de no haber jugadores el ranking sencillamente se mostrará vació.

Errores posibles y usos alternativos:

2.9. Consultar Estadísticas

Autor: Usuario
Comportamiento:

1. El usuario visualiza las siguientes estadísticas:
   - Número de partidas
   - Número de partidas ganadas
   - Número de partidas ganadas sin *hints*
   - Número de partidas ganadas sin *errores*
   - Número de partidas perdidas
   - Puntuación total del jugador
   - Puntuación media del jugador por partida ganada
   - Puntuación mejor en una partida
   - Tiempo medio por partida

   Por defecto todas las estadísticas empiezan a cero.

Errores posibles y usos alternativos:

2.10. Consultar Normas

Autor: Usuario, Usuario no registrado
Comportamiento:

1. El sistema muestra las reglas del juego.

Errores posibles y usos alternativos:

2.11. Configurar Ajustes

Autor: Usuario, Usuario no registrado
Comportamiento:

1. El sistema adapta la aplicación a las preferencias expresadas por el usuario.

Errores posibles y usos alternativos:

2.12. Salir

Autor: Usuario, Usuario no registrado
Comportamiento:

1. La aplicación se cierra.

Errores posibles y usos alternativos:

2.13. Jugar Partida

Autor: Usuario
Comportamiento:

1. El usuario selecciona el Kenken que desea jugar.

Errores posibles y usos alternativos:

2.14. Rellenar Casilla

Autor: Usuario
Comportamiento:

1. El usuario escoge qué casilla quiere rellenar y el número que quiere introducir. Este numero está entre 1 y N.
2. Si había notas en la casilla, estas se borran.
3. El sistema valida si el número introducido es correcto y en caso negativo muestra una animación y resta una vida.
4. Si la casilla es la última vacía del tablero y el número es correcto, se muestra una animación de victoria y se muestran estadísticas y puntuación de la partida.

Errores posibles y usos alternativos:

2.15. Borrar Casilla

Autor: Usuario
Comportamiento:

1. El usuario escoge que casilla quiere borrar.
2. El sistema vacía la casilla.

Errores posibles y usos alternativos:

2.1 Si la casilla ya estaba vacía no sucede nada.

2.16. Poner Nota

Autor: Usuario
Comportamiento:

1. El usuario selecciona la casilla donde desea poner la nota y el número que desea apuntar.
2. El sistema añade la nota nueva.

Errores posibles y usos alternativos:

2.1 Si el número seleccionado ya estaba anotado, se borra esa nota.

2.17. Pedir Hint

Autor: Usuario
Comportamiento:

1. El usuario muestra la voluntad manifiesta de pedir una pista.
2. El sistema rellena una casilla aleatoria vacía con el número correspondiente.

Errores posibles y usos alternativos:

2.1 Si ya se ha alcanzado el número máximo de pistas a utilizarse, el sistema no rellena ninguna casilla. Es decir, no ocurre nada.

2.2 Si la casilla rellenada era la ultima libre del tablero de juego, la partida termina.

2.18. Pausar Partida

Autor: Usuario
Precondición: El usuario ha expresado la voluntad de pausar la partida.
Comportamiento:

1. El sistema muestra un menú de pausa y para el temporizador.

Errores posibles y usos alternativos:

1.1 En caso de que el usuario quiera salir, se pasa al caso de uso *Reanudar Partida*.

2.19. Reanudar Partida

Autor: Usuario
Precondición: El usuario ha expresado la voluntad de reanudar una partida pausada.
Comportamiento:

1. El sistema sale del menú de pausa, vuelve a mostrar el Kenken con el progreso del usuario y reanuda el temporizador.

Errores posibles y usos alternativos:

2.20. Cerrar Partida

Autor: Usuario
Precondición El usuario ha manifestado la voluntad de cerrar la partida
Comportamiento:

1. El sistema guarda el progreso de la partida y vuelve al estado inicial del programa.

Errores posibles y usos alternativos:

2.21. Reiniciar Partida

Autor: Usuario
Precondición: El usuario profesa la voluntad de reiniciar la partida.
Comportamiento:

1. El sistema demanda la confirmación de la acción al usuario. El sistema devuelve al estado inicial todas las estadísticas del KenKen y borra todas las casillas.

Errores posibles y usos alternativos:

1.1 El usuario revoca la acción.

- El sistema devuelve al usuario al mismo punto en el que se encontraba antes de manifestar la voluntad de reiniciar el juego.

2.22. Continuar Partida

Autor: Usuario

Comportamiento:

1. El sistema presenta el último KenKen que se ha jugado y reanuda el contador de tiempo. También se restablecen todos los parámetros del KenKen a su estado anterior a pausar la partida.

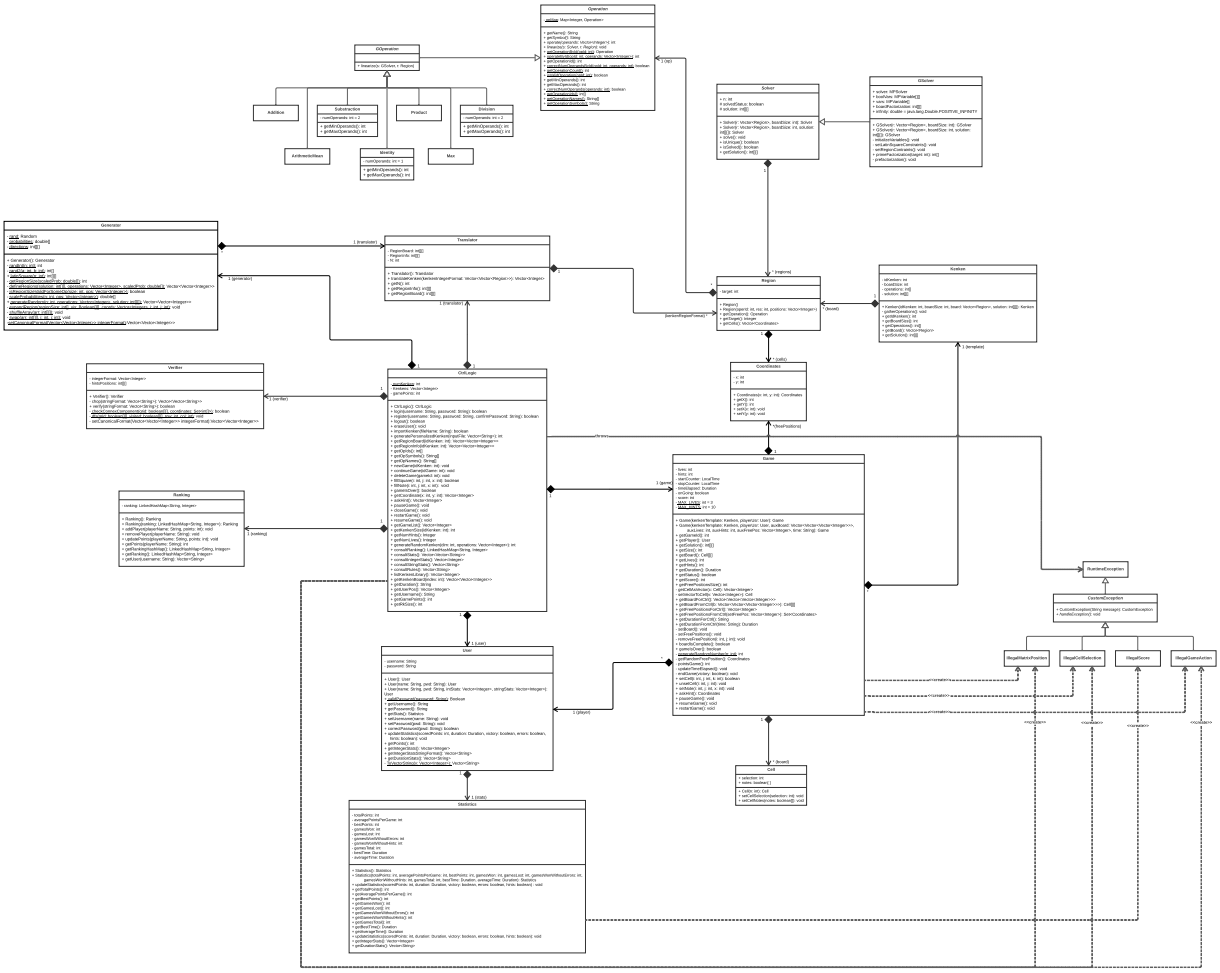
Errores posibles y usos alternativos:

1.1 Si el último KenKen ya se había terminado, el sistema vacía el tablero para que el usuario para volver a jugar el mismo KenKen.

Capítulo 3

Diagrama de clases

3.1. Diagrama de la capa de dominio



3.2. Descripción del diagrama de la capa de dominio

Coordinates

Atributos

- `int x`: coordenada vertical de una casilla (0 para la fila de arriba y $n-1$ para la fila de abajo).
- `int y`: coordenada horizontal de una casilla (0 para la columna de la izquierda y $n-1$ para la columna de la derecha).

Métodos

- `Coordinates(int x, int y)`: creadora.

Region

Atributos

- `Operation op`: operación de la región.
- `int target`: resultado de la región.
- `Vector <Coordinates> cells`: vector de coordenadas que define las casillas de la región.

Métodos

- `Region()`: creadora sin parámetros.
- `Region(int opId, int res, Vector<Integer> positions)`: creadora con parámetros.

Kenken

Atributos

- `int idKenken`: identificador único del Kenken.
- `int boardSize`: tamaño del Kenken.
- `int[] operations`: vector con los identificadores de las operaciones que contiene el Kenken.
- `Vector<Region> board`: vector de regiones que define el Kenken.
- `int solution[][]`: solución única del Kenken.

Métodos

- `Kenken(int idKenken, int boardSize, Vector<Region> board, int[][] solution)`: creadora.
- `void gatherOperations()`: mira qué operaciones hay en el Kenken y añade sus identificadores al vector `operations`.

Statistics

Atributos

- `int totalPoints`: total de puntos del usuario.
- `int averagePointsPerGame`: media de puntos por partida.
- `int bestPoints`: récord de puntuación en una partida.
- `int gamesWon`: partidas ganadas.
- `int gamesLost`: partidas perdidas.
- `int gamesWonWithoutErrors`: partidas ganadas sin errores.
- `int gamesWonWithoutHints`: partidas ganadas sin pistas.
- `int gamesTotal`: número de partidas acabadas.
- `Duration bestTime`: récord de tiempo de resolución de un Kenken.
- `Duration averageTime`: media de tiempo de resolución de Kenkens.

Métodos

- `Statistics()`: creadora sin parámetros (inicializa todo a 0).
- `Statistics(int totalPoints, int averagePointsPerGame, int bestPoints, int gamesWon, int gamesLost, int gamesWonWithoutErrors, int gamesWonWithoutHints, int gamesTotal, Duration bestTime, Duration averageTime)`: creadora con parámetros.
- `void updateStatistics(int scoredPoints, Duration duration, boolean victory, boolean errors, boolean hints)`: actualiza las estadísticas después de una partida acabada.
- `Vector<Integer> getIntegerStats()`: retorna un vector con las estadísticas que son enteros, en el mismo orden que en la creadora.

- `Vector<String> getDurationStats()`: retorna un vector con las dos estadísticas de tipo Duration, pasadas a String.

User

Atributos

- `String username`: nombre de usuario del usuario (es único y lo identifica).
- `String password`: contraseña del usuario.
- `Statistics stats`: estadísticas del usuario.

Métodos

- `User()`: creadora sin parámetros.
- `User(String name, String pwd)`: creadora con username y contraseña para cuando se registra un usuario (estadísticas a 0).
- `User(String name, String pwd, Vector>Integer> intStats, Vector<String> stringStats)`: creadora a partir de vectores para cargar el usuario de persistencia.
- `boolean validPassword(String password)`: retorna true si y solo si *password* cumple los requisitos para una contraseña, descritos en el caso de uso registrar perfil.
- `boolean correctPassword(String pwd)`: retorna true si y solo si el usuario tiene username y la contraseña coincide.
- `void updateStatistics(int scoredPoints, Duration duration, boolean victory, boolean errors, boolean hints)`: actualiza las estadísticas del jugador.
- `Vector<String> ToVectorString(Vector<Integer> v)`: retorna un vector de Strings con los enteros de `v`.

Game

Atributos

- `Kenken template`: Kenken asociado a la partida.
- `User player`: usuario que ha comenzado la partida.
- `Cell[][] board`: tablero de la partida, formado por celdas con su valor (el número que el jugador elige poner) y su vector de notas. En general, si el jugador aún no ha seleccionado ningún valor para la celda, `selection = 0`, mientras que a la que

`selection != 0`, todo el vector de notas estará en falso i.e. si hay selección, no hay notas (y las que había se borran).

- `int lives`: número de vidas que le quedan al usuario en la partida. Empieza con 3 y cada selección errónea le resta una vida.
- `int hints`: al iniciar el partido `hints = 0`; este valor se incrementa cada vez que el jugador pide una pista, hasta que llegue al número máximo de pistas permitidas, que se ha fijado a 10; entonces, cuando el jugador pida pistas, no se le dará ninguna.
- `Set<Coordinates> freePositions`: set con las casillas sin valor. Es decir, se inicializa con todas las casillas y la partida se acaba cuando este set está vacío. Esta estructura es necesaria cuando se quiera dar una pista al jugador, ya que se ha de elegir una posición aleatoria del tablero de la cual revelar la selección correcta.
- `LocalTime startCounter`: variable auxiliar para mantener actualizado el `timeElapsed`.
- `LocalTime stopCounter`: variable auxiliar para mantener actualizado el `timeElapsed`.
- `Duration timeElapsed`: contiene el tiempo que lleva jugando el usuario la partida.
- `boolean onGoing`: es true si y solo si la partida se está activamente jugando (por ejemplo, pasa a false cuando la partida se pausa o se termina).
- `int score`: representa la puntuación que ha conseguido el jugador con esta partida. Se actualiza solo cuando el juego haya acabado, y depende de varios factores: el número de errores cometidos y de pistas usadas y el tiempo total transcurrido. La puntuación total de un juego estará siempre entre 0 y 100.
- `int MAX_LIVES`: número máximo de vidas.
- `int MAX_HINTS`: número máximo de pistas.

Métodos

- `Game(Kenken kenkenTemplate, User playerUsr)`: creadora de un juego nuevo.
- `Game(Kenken kenkenTemplate, User playerUsr, Vector<Vector<Vector< Integer>>> auxBoard, int auxLives, int auxHints, Vector<Integer> auxFreePos, String time, boolean status)`: creadora de un juego recuperado de memoria.
- `Vector<Integer> getCellAsVector(Cell c)`: convierte un objeto del tipo `Cell` a un vector de *Integers*.
- `Cell setVectorToCell(Vector<Integer> v)`: convierte un vector de *Integers* a un objeto del tipo `Cell`.

- `Vector<Vector<Vector<Integer>>> getBoardForCtrl()`: convierte el parámetro de clase `board` en una matriz tridimensional de *Integers*.
- `Cell[][] getBoardFromCtrl(Vector<Vector<Vector<Integer>>> b)`: recupera el tablero de juego `board` de una matriz tridimensional de *Integers*.
- `Vector<Integer> getFreePositionsForCtrl()`: convierte el *set* de Coordenadas `freePositions` en un vector de *Integers*.
- `Set<Coordinates> getFreePositionsFromCtrl(Vector<Integer> setFreePos)`: recupera el *set* de Coordenadas `freePositions` de un vector de *Integers*.
- `String getDurationForCtrl()`: retorna la duración del juego en formato `String`
- `Duration getDurationFromCtrl(String time)`: recupera el `timeElapsed` del juego de un `String` en formato `Duration` ISO 8601.
- `void setBoard()`: inicializa el parámetro `board` con todos a cero.
- `void setFreePositions()`: inicializa el *set* de posiciones libres, insertando todas las posiciones del tablero.
- `void removeFreePosition(int i, int j)`: elimina la posición `(i,j)` del conjunto de posiciones libres.
- `boolean boardIsComplete()`: retorna cierto si el tablero está completo, es decir, todas las `selection` son correctas y el juego ya puede terminar; falso en caso contrario.
- `boolean gameIsOver()`: retorna cierto si el juego se ha terminado, lo cual ocurre si se han acabado las vidas del jugador o si se ha completado de forma correcta el tablero de juego.
- `int generateRandomNumber(int n)`: genera un numero aleatorio entre 0 (incluido) y n (excluido).
- `Coordinates getRandomFreePosition()`: devuelve una Coordenada aleatoria entre aquellas posiciones libres.
- `int pointsGame()`: calcula y retorna los puntos obtenidos al final del juego, teniendo en cuenta vidas perdidas, pistas utilizadas y tiempo empleado.
- `void updateTimeElapsed()`: actualiza el parámetro `timeElapsed`
- `void endGame(boolean victory)`: se llama al concluirse el juego, pasándole *true* si el usuario ha ganado, *false* en caso contrario; dentro se calculan los puntos totales y se actualizan las estadísticas del usuario.

- `boolean setCell(int i, int j, int k)`: añade el valor k a la posición (i, j) del tablero y devuelve cierto si este era el valor correcto y falso en caso contrario.
- `void unsetCell(int i, int j)`: elimina la selección de la posición (i, j) del tablero, si hay.
- `void setNote(int i, int j, int x)`: añade (si estaba) o elimina (si no estaba) la nota de valor x a la posición (i, j) del tablero.
- `Coordinates askHint()`: desvela un valor de la solución del tablero y devuelve la posición desvelada.
- `void pauseGame()`: pone en pausa el juego.
- `void resumeGame()`: resume el juego.
- `void stopGame()`: para el juego.
- `void restartGame()`: reinicia el juego desde cero.

Cell

Atributos

- `int selection`: valor que el usuario asigna a una casilla.
- `boolean[] notes`: vector de tamaño n donde que indica las notas que el usuario tiene puestas en una casilla.

Métodos

- `Cell(int n)`: creadora.

Verifier

Atributos

- `Vector<Vector<Integer>> integerFormat`: se guarda el formato estándar expresado cómo `Vector<Vector<Integer>>` eliminando las pistas.
- `Vector<Vector<Integer>> hintsPositions`: en cada vector se guarda el valor de la pista y las coordenadas de la casilla que refiere la pista.

Métodos

- `Verifier()`: creadora.

- `Vector<Vector<String>> chop (Vector<String> stringFormat)`: devuelve la entrada de la función con cada `String` del vector separado por espacios en un `Vector<Vector<String>>`.
- `boolean verify(Vector<String>)`: devuelve cierto si la entrada se corresponde con el formato estándar de la asignatura.
- `boolean checkConnexComponent(boolean[][] grid, Set<int[]> coordinates)`: devuelve cierto si el conjunto de coordenadas es conexo (accesible solo con movimientos verticales y horizontales).
- `void dfs(boolean[][] grid, boolean[][] visited, int row, int col)`: realiza un DFS en la matriz `grid` para comprobar si todas las casillas de la región son accesibles.
- `Vector<Vector<Integer>> getHintsPositions`: devuelve un vector con las posiciones de las pistas del formato estándar.
- `Vector<Vector<Integer>> getIntegerFormat()`: devuelve un vector de vector de enteros con el formato estándar canónico.
- `Vector<Vector<Integer>> setCanonicalFormat`: dado un vector de vector de enteros con el formato estándar devuelve un vector de vector de enteros con el formato estándar canónico. En el formato canónico las casillas de las regiones aparecen en el siguiente orden: de arriba a abajo y luego de izquierda a derecha, i.e, (1,1) es la "primeraçasilla y (3,3) la última. Las regiones se organizan según su casilla más "pequeñaz la primera región será la que tenga la casilla más pequeña entre las casillas de más pequeñas de cada región.

Translator

Atributos

- `Vector<Region> (kenken)RegionFormat`: guarda las regiones del posible KenKen ya en formato de la clase Region.
- `int N`: tamaño de la entrada a traducir (tamaño del tablero del posible Kenken).
- `int[][] RegionInfo`: array donde cada elemento es una array de dos elementos que guarda la operación y el resultado de una región.
- `int[][] RegionBoard`: matriz donde en cada posición se guarda en qué región pertence esa (el entero identifica a las regiones por orden de aparición en el input).

Métodos

- `Translator()`: creadora.
- `Vector<Region> translateKenken (Vector<Vector<Integer>> kenkenIntegerFormat)`: recibe un `Vector<Vector<Integer>>` que cumple el formato estándar de la asignatura y crea un vector de objetos de la clase `Region` con la información de la entrada.

Ranking

Atributos

- `LinkedHashMap<String, Integer> ranking`: se almacena el ranking de usuarios con su nombre y su puntuación.

Métodos

- `Ranking()`: creadora vacía.
- `Ranking(LinkedHashMap<String, Integer>)`: creadora con parámetro para asignar al atributo `ranking`.
- `void addPlayer(String playerName, int points)`: añade el usuario representado por el `playerName` al ranking con `points` puntos.
- `void removePlayer(String playerName)`: quita del ranking el usuario representado por `playerName`.
- `void updatePoints(String playerName, int points)`: pone los puntos del usuario `playerName` a `points`.
- `LinkedHashMap<String, Integer> getRanking()`: devuelve el ranking ordenado por puntuación y orden lexicográfico en caso de empate.
- `Vector<Integer> getUser (String username)`: devuelve la posición en el ranking del usuario con nombre *username*.
- `getRankingHashMap`: devuelve el `LinkedHashMap` donde se guarda el ranking.

Solver

Atributos

- `int n`: tamaño del potencial Kenken a resolver.
- `Vector<Region> regions`: vector de regiones que representa el potencial Kenken.

- `boolean solvedStatus`: es true si se ha resuelto exitosamente y false si no tiene solución.
- `int[][] solution`: una vez resuelto, contiene la solución del potencial Kenken.

Métodos

- `Solver(Vector<Region> r, int boardSize)`: creadora.
- `Solver(Vector<Region> r, int boardSize, int[][] solution)`: creadora con solución.
- `void solve()`: si el Kenken tiene solución, rellena `solution` con ella y pone `solvedStatus` a true. Si no, pone `solvedStatus` a false.
- `boolean isUnique()`: retorna true si y solo si la solución encontrada es única.

GSolver

Atributos

- `MPSolver solver`: solver de la librería de Google OR-Tools. En este se establecen las restricciones (inecuaciones) y finalmente se usa para resolver el sistema.
- `MPVariable[][][] boolVars`: variables booleanas del tipo x_{ijk} cuyo valor es 1 si y solo si la casilla (i,j) tiene un valor k.
- `MPVariable[][] vars`: variables enteras del tipo y_{ij} cuyo valor es el valor de la casilla (i,j).
- `int[][] boardFactorization`: cada fila i es la factorización en números primos de $i+1$, donde $i \in \{0, \ldots, n-1\}$. Dado que la factorización de $1, \ldots, n$ se usa repetidamente, esta se precalcula y se guarda en esta variable.
- `double infinity`: infinito, para definir las inecuaciones.

Métodos

- `GSolver(Vector<Region> r, int boardSize)`: creadora; crea el solver, las variables y precalcula la factorización de los números del 1 al n (que se guarda en `boardFactorization`).
- `GSolver(Vector<Region> r, int boardSize, int[][] solution)`: creadora; igual a la anterior, pero guarda la solución dada.
- `void initializeVariables()`: inicializa `boolVars` y `vars` como variables booleanas y enteras, respectivamente.

- `void setLatinSquareConstraints()`: define las restricciones/inecuaciones propias de un cuadrado latino, es decir:
  - Cada casilla contiene un solo valor.
  - Las filas no tienen valores repetidos.
  - Las columnas no tienen valores repetidos.
  - Si $x_{ijk} = 1$, entonces $y_{ij} = k + 1$.
- `void setRegionConstraints()`: define las restricciones/inecuaciones de cada región, en función de la operación.
- `int[] primeFactorization(int target)`: retorna un vector de tamaño n que representa la factorización en factores primos de target. Por ejemplo, si $n = 7$, sería [0,1,0,0,0,0,1].
- `void prefactorization()`: precalcula la factorización de $1, \ldots, n$ para reutilizarla (se guarda en `boardFactorization`).

Operation

Atributos

- `Map<Integer, Operation> opMap`: map estático que relaciona cada operación diferente con su identificador. Este contiene una instancia de cada una, que es la que se asigna siempre que se necesita una.

Métodos

- `String getName()`: retorna el nombre de una operación.
- `String getSymbol()`: retorna el símbolo de una operación.
- `int operate(Vector<Integer> operands)`: retorna el resultado de aplicar la operación a los elementos del vector.
- `void linearize(Solver s, Region r)`: define en el solver las restricciones que impone la operación en la región r.
- `Operation getOperationById(int opId)`: retorna una instancia de la operación que corresponda según el identificador `opId`.
- `int operateById(int opId, Vector<Integer> operands)`: retorna el resultado de aplicar la operación `opId` a los elementos del vector.
- `int getOperationId()`: retorna la operation ID del objeto.

- `boolean correctNumOperandsById(int opId, int operands)`: retorna true si y solo si *operands* es un número válido de operandos para la operación con ID = *opId*.
- `int getOperationCount()`: retorna el número total de operaciones.
- `boolean isValidOperation(int opId)`: retorna true si y solo si existe una operación con ID = *opId*.
- `int getMinOperands()`: retorna el número mínimo de operandos de la operación (por defecto 2).
- `int getMaxOperands()`: retorna el número máximo de operandos de la operación (por defecto `Integer.MAX_VALUE`).
- `boolean correctNumOperands(int operands)`: retorna true si y solo si *operands* es un número válido de operandos para la operación.
- `int[] getOperationIds()`: retorna un array con los identificadores de las operaciones existentes.
- `String[] getOperationNames()`: retorna un array con los nombres de las operaciones existentes.
- `String[] getOperationSymbols()`: retorna un array con los símbolos de las operaciones existentes.

GOperation

Métodos

- `void linearize(GSolver s, Region r)`: método de linealización para el caso de usar el GSolver.

CustomException

Métodos

- `CustomException(String message)`: creadora.
- `void handleException()`: define la acción que se lleva a cabo para resolver.[el] problema.

CtrlLogic

Atributos

- `int numKenken`: número de KenKens guardados.

- `vector<int> Kenkens`: un vector que contiene en la posición i-ésima el tamaño del kenken i.
- `User user`: usuario conectado.
- `Game game`: instancia de la partida en juego.
- `Verifier verifier`: instancia del verificador de formato de las entradas.
- `Translator translator`: instancia del traductor de entradas verificadas al formato `Vector<Region>`.
- `CtrlData ctrlData`: instancia del controlador de la capa de persistencia.
- `Ranking ranking`: instancia del ranking de jugadores.
- `int gamePoints`: puntos obtenidos en la partida que acaba de terminar, cuando la partida está en juego su valor es 0.

Métodos

- `CtrlLogic()`: creadora.
- `boolean login(String username, String password)`: loggea un usuario ya existente.
- `boolean register(String username, String password, String confirmPassword)`: crea un nuevo usuario que no contenga un espacio con la contraseña `password` si:
  - `username` no es el `String` vacío.
  - `password` es igual a `confirmPassword`.
  - `validPassword(password)` devuelve `true`.
  - No existe ningún usuario con nombre de usuario `username`.
- `boolean logout()`: cierra la sesión del usuario actual (pone a `user` = `null`).
- `void eraseUser()`: borra el usuario conectado de persistencia, del ranking y pone `user` = `null`.
- `boolean importKenken(String fileName)`: lleva a cabo la *pipeline* completa de importar un Kenken del fichero `fileName`, que se constituye de:
  - Leer el contenido del fichero `fileName` y guardarlo en un `Vector<String>` por líneas.
  - Pasar el verificador a este vector.

- En caso de tener formato válido, traducir la entrada a un `Vector<Region>`.
- Mandar al Solver el `Vector<Region>`, para que compruebe si tiene solución y es única y la calcule.
- Compruebe si las pistas dadas en el fichero de entrada se corresponden con la solución del Solver.
- En caso que todo el proceso haya funcionado se guarda en persistencia como KenKen.

Devuelve `true` si todo el proceso anterior funciona con éxito. `false` en caso contrario.

- `int generatePersonalizedKenken(Vector<String> inputFile)`: Ejecuta la *pipeline* completa de importar un Kenken, sin embargo, en esta ocasión a partir del formato estándar en *String* que obtiene desde la capa de presentación. Devuelve el identificador del nuevo Kenken, -1 si el input no es un Kenken.
- `Vector<Vector<Integer>> getRegionBoard(int idKenken)`: *Getter* del atributo *RegionBoard* que se guarda en la persistencia de un Kenken .
- `Vector<Vector<Integer>> getRegionInfo(int idKenken)`: *Getter* del atributo *getRegionInfo* que se guarda en la persistencia de un Kenken.
- `int[] getOpIds()`: *Getter* del vector de los identificadores de las operaciones.
- `String[] getOpSymbols()`: *Getter* del vector de los símbolos de las operaciones.
- `String[] getOpNames()`: *Getter* del vector de los nombres de las operaciones.
- `void newGame(int idKenken)`: si hay una partida en curso la salva en persistencia. Comprueba que no exista ya una partida con el Kenken asociado a `idKenKen` y `user`. En caso que no lo haya, se lee de persistencia todo lo necesario para instanciar un objeto `KenKen` y genera una nueva partida asociando el `Kenken` creado y `user` y se establece como partida actual (se asigna a `game`). Se actualiza el contador de partidas.
- `void continueGame(int idGame)`: si hay una partida en curso la guarda en persistencia. Se lee de persistencia el tablero del KenKen asociado a la partida representada por `idGame`. Se instancia un objeto KenKen. Se leen de persistencia los diferentes parámetros asociados a la partida `idGame` y se instancia una partida con ellos, `user` y el KenKen creado anteriormente. Se asocia esta partida a `game` y se pone `currentGame` a `idGame`.
- `void deleteGame(int gameId)`: se borra la partida con identificador gameId del usuario actual.

- `void fillSquare(int i, int j, int x)`: se rellena la casilla (i, j) del tablero asociado a `game` con el valor x. Si este resultado es incorrecto y con este error se acaba la partida, se borra esta de persistencia y se pone `game=null`.
- `void eraseSquare(int i, int j)`: borra el valor de la casilla (i, j) del tablero de *game* (este trabajo lo hace la clase `Game`).
- `void fillNote(int i, int j, int x)`: asigna una nota a la casilla (i, j) del tablero de *game* con el valor x.
- `Vector<Integer> getCoordinate(int x, int y)`: devuelve el contenido de la casilla (i, j) de la partida actual.
- `void askHint()`: pide una pista de la partida asociada al `game`. Una pista consiste en la resolución de una casilla aleatoria vacía de la partida.
- `void pauseGame()`: pausa la partida. Quiere decir pausar el cronómetro aunque está acción la hace la clase `Game`.
- `void closeGame()`: si `game` no es `null` se guarda la partida en persistencia y se pone `game = null`.
- `void restartGame()`: reinicia la partida, es decir llama a un método de la clase `Game` que reinicia los parámetros de la partida
- `void resumeGame()`: continúa una partida pausada, es decir el objeto `game` llama a una función que vuelve a encender el cronometro.
- `Vector<Integer> getGameList()`: devuelve un vector que contiene en la i-ésima posición el identificador del kenken de la i-ésima partida del jugador actual.
- `int getKenkenSize(int idKenken)`: devuelve el tamaño del kenken con identificador idKenken.
- `Integer getNumHints()`: devuelve el número de pistas disponibles en la partida actual.
- `Integer getNumLives()`: devuelve el número de vidas disponibles en la partida actual.
- `int generateRandomKenken(int dim, Vector<Integer> operations)`: genera un KenKen aleatorio con el tamaño indicado por *dim* y con la posibilidad de tener las operaciones que figuran en el vector *operations*. Devuelve el identificar del nuevo kenken, -1 si no se ha podido generar un kenken con las características solicitadas.
- `LinkedHashMap<String,Integer> consultRanking()`: devuelve el *ranking* ordenado por puntos y en caso de empate se ordena lexicográficamente.

- `Vector<String> consultStats()`: consulta las estadísticas del usuario `user`.
- `Vector<Integer> consultIntegerStats()`: *Getter* de las estadísticas numéricas del usuario.
- `Vector<String> consultStringStats()`: *Getter* de las estadísticas en formato *String* del usuario.
- `Vector<String> consultRules()`: lee las normas de persistencia.
- `Vector<Integer> listKenkenLibrary()`: devuelve el atributo *Kenkens* de la clase.
- `Vector<Vector<Integer>> getKenkenBoard(int index)`: devuelve el tablero del kenken con identificador *index*.
- `String getDuration()`: devuelve el tiempo de la partida actual, si es *null* entonces devuelve 0.
- `Vector<Integer> getUserPos()`: devuelve la posición del usuario en el *ranking*.
- `String getUsername()`: devuelve el nombre del usuario actual.
- `int getRkSize()`: devuelve el tamaño del ranking.

Generator

Atributos

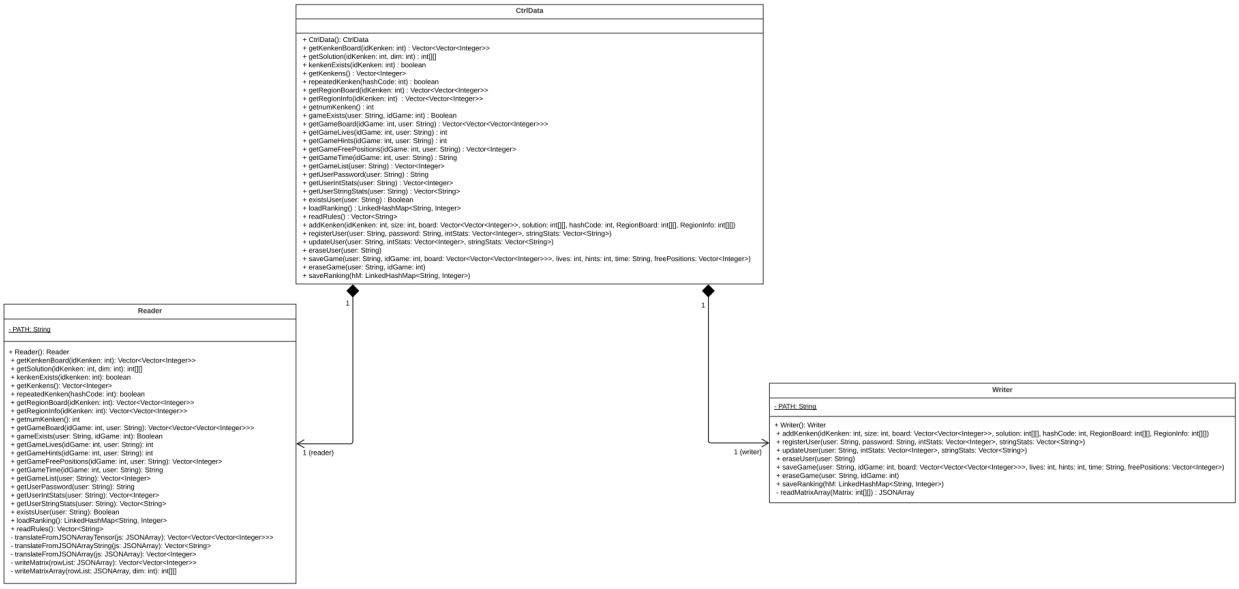
- `Random rand`: parámetro auxiliar para generar números aleatorios.
- `Translator translator`: instancia de un traductor para generar un vector de regiones a partir del formato estándar.
- `double[] probabilities`: array de tamaño 7 donde cada posición i corresponde a la probabilidad inicial de que toque una región de tamaño i cada vez que se intenta generar una. Estas probabilidades se modifican dinámicamente según las características del Kenken a generar, pero siempre se parte de este estado inicial.
- `int[][] directions`: estructura auxiliar que contiene las 4 direcciones $(-1, 0), (1, 0), (0, -1), (0, 1)$. Estas se usan para expandir las regiones en direcciones aleatorias.
- `Vector<Vector<Integer>> setCanonicalFormat`: dado un vector de vector de enteros con el formato estándar devuelve un vector de vector de enteros con el formato estándar canónico. En el formato canónico las casillas de las regiones aparecen en el siguiente orden: de arriba a abajo y luego de izquierda a derecha, i.e, (1,1) es la "primeraçasilla y (3,3) la última. Las regiones se organizan según su casilla más "pequeñaz

la primera región será la que tenga la casilla más pequeña entre las casillas de más pequeñas de cada región.

Métodos

- `Generator()`: creadora vacía
- `int randInt(int n)`: genera un número aleatorio entre 0 y $n-1$ usando el atributo `rand`.
- `int[] rand2(int a, int b)`: retorna un array con los valores a y b ordenado aleatoriamente.
- `int[][] latinSquare(int n)`: genera un Latin Square aleatorio de tamaño $n \times n$.
- `int getRegionSize(double[] scaledProb)`: retorna un entero entre 1 y 7, teniendo en cuenta las probabilidades de `scaledProb`.
- `Vector<Vector<Integer>> defineRegions(int[][] solution, Vector<Integer> operations, double[] scaledProb)`: define un potencial Kenken a partir de la solución (Latin Square), las operaciones a usar y las probabilidades de los tamaños de las regiones.
- `boolean isRegionSizeValidForSomeOp(int size, Vector<Integer> ops)`: retorna true si el tamaño `size` es válido para alguna de las operaciones del Kenken que se está generando.
- `double[] scaleProbabilities(int n, Vector<Integer> ops)`: retorna un nuevo array de probabilidades escalado, a partir de las probabilidades iniciales y de las operaciones del Kenken a generar.
- `Vector<Vector<Integer>> generateRandom(int n, Vector<Integer> operations, int[][] solution)`: retorna un Kenken válido y con solución única de tamaño $n \times n$, usando solo las operaciones de `operations` y que tiene `solution` como solución única. Si no ha podido generarlo en 200 iteraciones, retorna `null`.
- `void expandRegion(int[] regionSize, Boolean[][] vis, Vector<Integer> coords, int i, int j)`: método recursivo para construir las regiones de tamaño `regionSize`, moviéndose en direcciones aleatorias.
- `void shuffleArray(int[][] arr)`: reordena aleatoriamente las filas de un array del tipo `int[][]`.
- `void swap(int[][] arr, int i, int j)`: intercambia las filas i y j del array `arr`.

3.3. Diagrama de la capa de persistencia



3.4. Descripción del diagrama de la capa de persistencia

CtrlData

Descripción

CtrlData es el controlador de la capa de persistencia. Se encarga de gestionar las llamadas a persistencia entre la capa de dominio y los ficheros de persistencia del proyecto: Game.json, Kenken.json y Ranking.json, User.json. Todos estos ficheros contienen información relacionada con las clases que les dan nombre.

Reader

Descripción

La clase Reader contiene los métodos que leen de los ficheros de persistencia de las cuatro clases que lo necesitan, así como del fichero de texto que contiene las reglas. En general, se encarga de traducir el formato que utilizan los ficheros json a estructuras de datos de *strings* y *ints*. Cuenta con las siguientes funcionalidades:

- `Kenken`: Esta clase cuenta con algunos *getters* de sus atributos y diversos métodos que permiten:
    - Comprobar si existe un *Kenken* con un cierto identificador.

    - Obtener un vector cuya posición i-essima contiene el tamaño del *Kenken* con identificador i.
    - Comprobar si añadir un *Kenken* supondría repetirlo en la persistencia.
    - Obtener el número de *Kenkens* que hay en persistencia.
- `User`: Esta clase cuenta con algunos *getters* de sus atributos y un método que permite comprobar si existe un usuario con un cierto nombre de usuario.
- `Game`: Esta clase cuenta con algunos *getters* de sus atributos y diversos métodos que permiten:
    - Comprobar si existe una partida dado un identificador de *Kenken* y un nombre de usuario.
    - A partir de un nombre de usuario, obtener un vector cuya posición i-essima contiene el identificador del *Kenken* de la i-essima partida de dicho usuario.
- `Ranking`: Del *ranking* solo tenemos un método que se encarga de instanciar un *LinkedHashMap <String,Integer>* a partir de la persistencia.

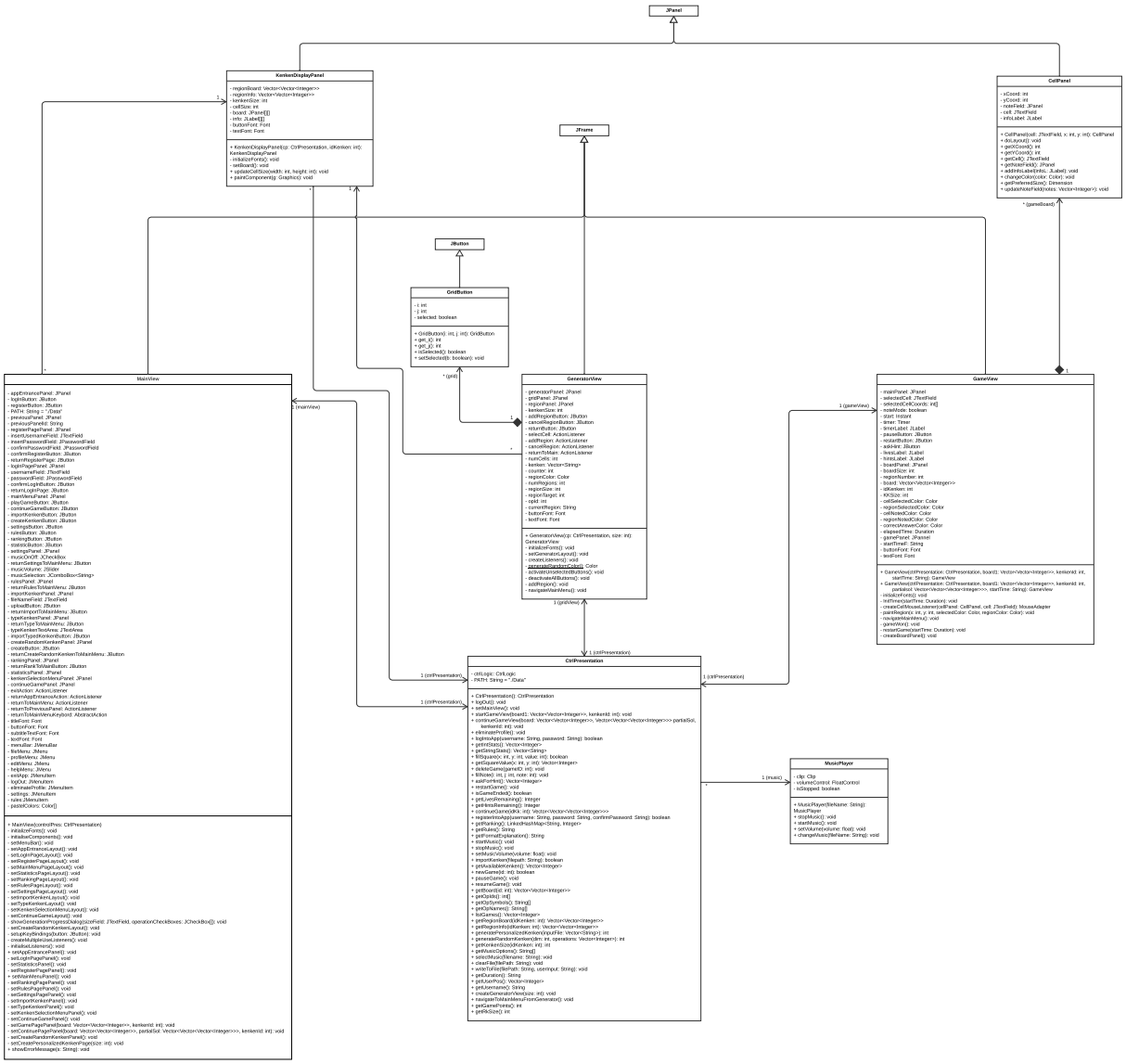
Writer

Descripción

La clase Writer contiene los métodos que modifican los ficheros de persistencia de las cuatro clases que lo necesitan. Se encarga de traducir estructuras de datos de *strings* y *ints* al formato que requieran los ficheros json. Cuenta con las siguientes funcionalidades:

- `Kenken`: Nosotros trabajamos bajo la asunción de que los *Kenkens* no se pueden repetir ni borrar, por tanto, nos basta con un método para añadir *Kenkens* que tiene por prerrequisito que no haya otro *Kenken* con el mismo *Hash* que funciona de identificador único.
- `User`: Para los usuarios contamos con tres métodos: uno que registra al usuario, uno que actualiza sus estadísticas y otro que lo elimina de la persistencia.
- `Game`: Las partidas cuentan con dos métodos: uno que guarda la partida y otro que la elimina de la persistencia. Cabe recalcar que para nosotros una partida está inequívocamente definida por un usuario y un *Kenken*, por lo que se ha construido persistencia de forma que no sea posible que un usuario tenga a la vez dos partidas con un mismo *Kenken*.
- `Ranking`: Del *ranking* solo tenemos un método que se encarga de guardarlo en persistencia.

3.5. Diagrama de la capa de presentación



3.6. Descripción del diagrama de la capa de presentación

CtrlPresentation

Descripción

`CtrlPresentation` es el controlador de la capa de presentación. Este controlador principalmente cumple la función de relacionar la capa de dominio con la de presentación, siendo la clase que se comunica con el controlador de lógica para poder subministrar la información necesaria en presentación para que se pueda operar la aplicación con éxito. Además también es el encargado de comunicar la vista principal con la clase encargada de reproducir música

en la aplicación.

Además el controlador es el encargado de gestionar el cambio de vistas cuando sea necesario pues es el que tiene como atributo las instancias de dichas vistas.

MainView

Descripción

`MainView` es la clase más grande en líneas de código de la capa de presentación, extiende la clase `JFrame` de `java.swing`. Principalmente esta vista es la encargada de la navegabilidad por todos los paneles de la aplicación a excepción del panel de juego y de la vista de generar un Kenken de forma personalizada. Esta clase tiene como atributos los diferentes paneles para el funcionamiento de la aplicación, cada uno con sus elementos pertinentes. Esta clase también es la encargada de definir y generar el *layout* de cada uno de los paneles. Además también gestiona el cambio entre los diferentes paneles. Esta clase también setea las fuentes de texto especiales para la aplicación.

Más en general, las funcionalidades de la aplicación que se incluyen en esta vista son:

- El incio de sesión.
- El registro de usuarios.
- La navegación por el menú principal de la aplicación.
- El acceso a la biblioteca de Kenken's para inciar una nueva partida.
- El inicio de una nueva partida.
- El acceso y navegación del registro de partidas empezadas por un usuario.
- La posibilidad de continuar una partida inciada. En caso de querer continuar o iniciar una partida se le pasa al controlador de presentación los datos necesarios para que este (el controlador) cree la vista de la partida y haga el cambia de vistas. Así evitamos acoplamiento entre vistas.
- La importación vía fichero de texto (.txt) de un potencial Kenken.
- La importación vía texto escrito de un potencial Kenken.
- La creación de un Kenken de forma aleatoria.
- El acceso a la vista de creación personalizada de Kenken's. En esta acción se llama al controlador de presentación para que sea él quien haga el cambio de vistas. Así evitamos acoplamiento entre vistas.
- La consulta del ranking.

- La consulta de las estadísticas de usuario.
- La consulta y modificación de los ajustes de la aplicación.

GameView

Descripción

`GameView` es la clase que contiene la vista de la experiencia de juego, es una clase que extiende la `JFrame` de `java.swing`. Esta vista está compuesta de un tablero NxN casillas seleccionables con las regiones marcadas (que es el Kenken a rellenar) y dos menús de botones: N botones para poder introducir números en las casillas del Kenken y un menú de botones para poder pausar y salir, reiniciar el juego y poder pedir pistas. Destacar de esta clase que se usa una clase privada que extiende la clase `JPanel` de la librería estándar de `java.swing`, llamada `CellPanel`. Esto se debe a qué realizamos modificaciones a la base de la clase para que cada panel pudiera almacenar toda la información que necesita una celda y tuviera los métodos adaptados al juego. Notar que esta clase tambíen setea las fuentes mismas personalizadas que hemos comentado en `MainView`.

La decisión de realizar una vista separada para la experiencia de juego creemos que es acertada, ya que así podemos compartimentar el código y a nivel de diseño de juego tiene sentido separar los menús del juego por sí.

Esta clase se comunica constantemente con el controlador para ir actualizando los cambios que hace el jugador en la interfaz en la lógica del juego y viceversa. También se comunica con el controlador cuando se quiere volver al menú inicial, avisándolo para que este haga el cambio de vistas. De esta forma conseguimos que no esté acoplada con ninguna de las otras vistas.

GeneratorView

Descripción

Esta clase contiene la vista de la generación de Kenken's manual, extiende la clase `JFrame` de `java.swing`..

Destacar que en esta clase se usa una clase privada (`GirdButton`) que extiende la classe `JButton` añadiendo dos atributos para almacenar en cada JButton sus coordenadas en la matriz donde se crea el Kenken. También que esta clase también setea las fuentes especiales ya comentadas.

Esta vista consta de una matriz de `GridButtons`'s de tamaño N (donde N es el tamaño del Kenken a crear) y de un botón que abre un menú donde se puede seleccionar las carac-

terísticas de la región que se quiere introducir en el Kenken. Entonces la matriz de botones se vuelve interactuable para poder seleccionar que casillas formaran parte de la región que se ha descrito en el menú.

Esta clase se comunica con el controlador de presentación para poder pasarle la propuesta de Kenken realizada por el usuario y que este lo pase a lógica para que pase por la línea de importación de Kenken's y, en caso de no presentar problemas, se añada a la biblioteca. También se comunica con el controlador cuando se quiere volver al menú inicial, avisándolo para que este haga el cambio de vistas. De esta forma conseguimos que no esté acoplada con ninguna de las otras vistas.

Consideramos necesario tener una nueva vista para tener el código de `MainView` más limpio y, además, al ser una funcionalidad bastante más elaborada que navegar un menú pensamos que tenía sentido que tuviera su propia vista.

MusicPlayer

Descripción

Esta clase usando la librería de `javax.sound` se encarga de gestionar la reproducción de la música de fondo de la aplicación. La clase `MainView` accede a las funcionalidades de `MusicPlayer` a través del controlador para evitar acoplamientos.

KenkenDisplayPanel

Descripción

Esta clase que extiende de la clase `JPanel` que se usa para poder mostrar una previsualización del tablero de los Kenken que se generan de forma aleatoria en la aplicación.

Capítulo 4

Estructuras de datos y algoritmos

4.1. Resolución de Kenkens

En este capítulo se describe la *pipeline* de resolución de Kenkens, haciendo énfasis en las estructuras de datos y algoritmos.

4.1.1. Verificador

Al recibir un potencial Kenken, este primero se pasa por el Verificador (clase `Verifier.java`). Esta clase se encarga de comprobar que la entrada recibida tiene formato estándar de Kenken. Es decir, descarta entradas que tengan formatos diferentes del estándar, que usen operaciones que no son válidas, etc.

De esta manera, si una entrada ha pasado con éxito el verificador, lo único que queda comprobar para ver que efectivamente se trata de un Kenken, es ver que tiene una única solución.

Para más detalles de los algoritmos que usa el Verificador, ver el apéndice A.

4.1.2. Traductor

Después del Verificador, la entrada se pasa al Traductor, que se encarga de traducir el `Vector<String>` que representa la entrada, a un `Vector<Region>`, para poder usar el Solver.

4.1.3. Solver

El Solver se encarga de comprobar que la entrada, que ya sabemos que tiene estructura de Kenken, tiene una única solución. Así, básicamente se divide en dos funcionalidades: encontrar una solución y comprobar que es única.

Al inicio del proyecto, pasamos bastante tiempo discutiendo cómo íbamos a plantear la resolución de Kenkens. Después de hacer la pertinente investigación de cómo se suelen resolver los Kenkens, rápidamente vimos que lo más típico era hacer un algoritmo de `backtracking`, suficientemente optimizado para que sea eficiente. Sin embargo, en nuestro equipo, dada nuestra predilección por las matemáticas, decidimos explorar el camino más numérico.

La idea que perseguíamos es la de hacer una traducción de un Kenken a un modelo matemático que podamos tratar y resolver. Este planteamiento viene de manera natural a partir de la definición de un Kenken, ya que restricciones como la de la suma se pueden expresar como ecuaciones lineales donde las variables son los valores de cada casilla. Sin embargo, esta traducción no es trivial, pues por un lado hay que modelar las restricciones de *latin square*, y por otro tenemos operaciones que dan lugar a ecuaciones no lineales que pueden dar problemas.

Para hacer este proceso, nos hemos basado en *An Integer Programming Model for the KenKen Problem* [1]. En este paper se explica cómo expresar matemáticamente las restricciones de un KenKen clásico (con $+, -, *, /$) y propone formas de linealizar estas operaciones.

El planteamiento se resume en definir las siguientes variables enteras:

$$y_{ij} = k \iff \text{la casilla (i,j) tiene el valor k}$$

Y las siguientes variables binarias:

$$x_{ijk} = \begin{cases} 1 : \text{si la casilla (i,j) tiene valor k} \\ 0 : \text{si no} \end{cases}$$

Que se relacionan a través de la siguiente restricción:

$$\sum_{k=1}^{n} k \cdot x_{ijk} = y_{ij}$$

Tener ambos conjuntos de variables se justifica con que, depende del caso, unas variables son más convenientes que las otras.

Por tanto, la idea consiste en pasar del KenKen a un sistema de inecuaciones de variables enteras y variables binarias. Una vez tenemos el KenKen modelado, podemos usar alguna de las muchas librerías que te permiten solucionar este tipo de sistemas. A partir de aquí obtenemos la solución del KenKen, si es que esta existe.

En este punto, toca plantearse como afrontar el problema de la unicidad. Para hacerlo, dada una solución, podemos añadir una restricción que sea "la solución no puede ser esta". De esta manera, la solución será única si y solo si el problema resultante es inviable. Formalmente, dada una solución $\hat{x}$, si definimos $S = \{(i, j, k) : \hat{x}_{ijk} = 1\}$, la restricción es:

$$\sum_{(i,j,k) \in S} (1 - x_{ijk}) \geq 1$$

Fundamentalmente estamos restringiendo que al menos una de las variables booleanas pasen de 1 a 0 para forzar que la solución sea diferente de la que ya tenemos.

En cuanto a la librería para resolver el sistema de inecuaciones, hemos optado por *Google OR-Tools* [2]. Esta es una librería *open-source* desarrollada por Google para resolver problemas de *linear programming* (LP), *mixed integer programming* (MIP) y *constraint programming* (CP), entre otros problemas de optimización.

4.1.4. Kenken

Llegados a este punto de la *pipeline*, ya sabemos que la entrada era un Kenken, y por lo tanto, podemos guardarlo en persistencia para posteriormente ser jugado. Lo guardamos junto con la solución, pues esta es única y ya hemos hecho el esfuerza de encontrarla.

Ahora, cuando el usuario quiere comenzar una partida con ese Kenken (o cualquiera de los que hay en persistencia), se instancia un objeto de la clase Kenken que es usado por partida. En particular, este objeto contiene la solución, de manera que comprobar que el usuario va poniendo los números de manera correcta, se hace en tiempo constante.

Para instanciar un Kenken, se carga de persistencia el Kenken en formato estándar y la solución guardada. Se usa el Traductor para pasar del formato estándar a un vector de regiones (ahora no hay que pasar por el Verificador ni el Solver, pues ya ha sido todo verificado previamente). Con esto, obtenemos todo lo que necesita la creadora de Kenken para obtener el objeto.

4.1.5. Observaciones

Gestión de las pistas en una entrada

Sabemos que, como parte del formato estándar, a continuación de cada casilla podemos tener entre corchetes la solución de esa casilla. Lo que hacemos en ese caso es añadir la restricción de que la solución encontrada, a parte de ser única, tiene que coincidir con todas las pistas que lleguen de la entrada. Es decir, se puede dar el caso en que se introduzca un Kenken válido y con solución única, pero si las pistas no forman parte de la solución correcta, el Kenken es rechazado por el sistema. Esta ha sido una decisión nuestra, pues no venía explicitado, y podría ser cambiada con facilidad.

Para hacerlo, la *pipeline* está adaptada para leer esta entrada e identificar que es válida. El Solver actualmente no tiene en cuenta las pistas: encuentre la solución única y es el CtrlLogic el que compara la solución correcta (si es que existe) con las pistas ofrecidas en la entrada. Tal y como está planteado, una posible optimización sería usar las pistas para resolver el Kenken más rápido, ya que ahora mismo se rechaza el Kenken si no coincide con las pistas.

Sin embargo, de esta manera se podría decidir aceptar el Kenken pese a que las pistas no coincidan con la solución correcta. Por ejemplo, se le podría consultar al usuario, si quiere guardar el Kenken, teniendo en cuenta que efectivamente es un Kenken pero que las pistas no coinciden con solución correcta.

Operaciones

De cara a añadir nuevas operaciones, se fuerza a implementar un método `void linearize( Solver s, Region r)` que añade al Solver las restricciones de la operación en cuestión a la región r. Esto es así porque nuestro programa resuelve los Kenkens como un sistema de inecuaciones.

Ahora bien, nuestra intención es tener encapsulada nuestra implementación, que usa la librería de Google OR-Tools. Para ello, la clase Solver es abstracta y obliga a tener los métodos públicos que el CtrlLogic necesita para la resolución de Kenkens: `solve(), isUnique(), isSolved(), getSolution()`. Así, nuestra implementación del Solver está en la clase GSolver (Google Solver), que esta sí usa la librería mencionada. De esta manera, si se quiere usar otra librería, bastaría con crear una clase para esta que implemente los métodos necesarios para resolver el Kenken.

De manera similar, tenemos una clase abstracta Operation. Hemos creado una subclase también abstracta GOperation que implementa la función de linealización cuando el Solver es un GSolver. Todas las operaciones que hemos definido son GOperaciones, pues la linealización está hecha con el solver de la librería usada.

En caso de querer usar otra librería, se puede crear una subclase abstracta de Operation y crear subclases con cada operación, implementadas de manera adecuada según el solver de terceros que se use.

Time complexity

Dada la naturaleza de los problemas de optimización, es difícil determinar la complejidad en tiempo del Solver. Actualmente estamos usando *BOP* como solver. Este utiliza diferentes estrategias en función de las características del problema. Por ejemplo, técnicas como *Local Search (LS)*, *Large Neighborhood Search (LNS)* o *Linear Programming (LP)*.

El rendimiento depende en gran parte de la estructura del problema. Por ejemplo, la densidad de la matriz de restricciones puede impactar en gran medida el tiempo de computación.

En la práctica, la complejidad puede ser influenciada por muchos factores, por lo que típicamente se describe en términos de rendimiento empírico en lugar de usar *O-notation*.

Lo que sí podemos decir es que la fase inicial en la que se inicializan las variables y restricciones tiene una complejidad polinómica, y la fase de búsqueda de solución (función *solve()*) tiene un comportamiento exponencial.

4.2. Generación de Kenkens

En esta sección se explica la funcionalidad de generar Kenkens aleatorios.

4.2.1. Generar solución

El primer paso para la generación de un nuevo Kenken es generar la que será la solución. Este proceso se resume en generar un cuadrado latino, es decir, una matriz de $n \times n$ elementos en la que cada casilla está ocupada por uno de los n símbolos de tal modo que cada uno de ellos aparece exactamente una vez en cada columna y en cada fila.

Una manera *naive* de implementar esto sería una especie de backtracking: comenzamos con una matriz vacía y rellenamos la primera fila con los números del 1 al n ordenados aleatoriamente. Vamos rellenando las siguientes filas, comprobando en cada caso que se cumplan las restricciones. Si llegamos a un punto donde no podemos continuar, hacemos backtracking hasta llegar a una combinación válida.

Sin embargo, investigando un poco, hemos dado con el famoso algoritmo de Jacobson y Matthews [3]. A continuación explicamos de manera resumida como funciona el algoritmo:

Cubo de incidencia

En primer lugar, definimos un cubo de incidencia como un cubo $n \times n \times n$ cuyos ejes corresponden a las filas, columnas y números del cuadrado latino. Es decir, se cumple que:

$$\texttt{incidenceCube}(i, j, k) = 1 \iff \texttt{latinSquare}(i, j) = k$$

Para preservar la condición de cuadrado latino, todo eje ha de tener una celda con valor 1 y el resto tener valor 0. Esto implica que si fijamos 2 coordenadas, la suma de toda linea ha de sumar 1. Es decir,

$$\sum_{i=1}^{n} \texttt{cube}(i, j_{fixed}, k_{fixed}) = \sum_{j=1}^{n} \texttt{cube}(i_{fixed}, j, k_{fixed}) = \sum_{k=1}^{n} \texttt{cube}(i_{fixed}, j_{fixed}, k) = 1$$

Descripción de los movimientos

Un ± 1-movimiento consiste en sumar o restar 1 a ciertos elementos del cubo de manera que no se altere la suma anterior. Pero esto puede dar lugar a lineas con dos valores 1 y un valor -1. Cuando esta situación ocurre, decimos que el cubo es *impropio*. Si no, el cubo es *propio*. El movimiento a hacer depende de si partimos de un cubo propio o impropio:

1. **Propio:** se escoge aleatoriamente una celda con valor 0 entre las $n^2(n-1)$ posibles (hay n^2 celdas con valor 1). Si denominamos $t = (t.x, t.y, t.z)$ a esta celda, esta nos define 3 lineas (una en cada eje) que contienen una sola celda con valor 1. Esto nos determina un subcubo $2 \times 2 \times 2$, generado por t y las 3 celdas que contienen los 1s. A estas 8 celdas, se suma o resta 1, según la figura. Si despues de las operaciones, la celda opuesta a t tiene valor -1, el nuevo cubo es impropio. Si no, es propio.

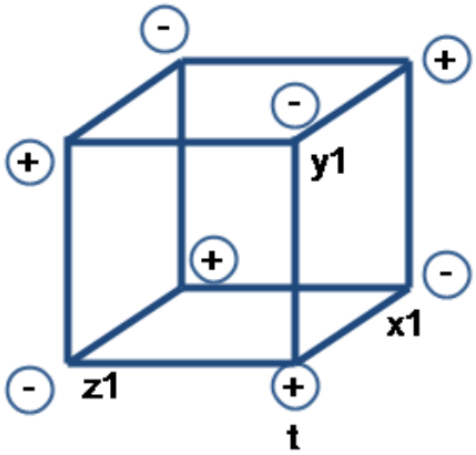


Figura 4.1: Subcubo $2 \times 2 \times 2$ de un ± 1-movimiento. Extraído de [5].

2. **Impropio:** ahora la celda t es la que contiene el valor -1 (solo hay una en el cubo). Como cada linea suma 1, la fila de t contiene un -1 en $t.x$ y otras dos celdas con valor 1. Se escoge una de estas dos aleatoriamente para ser uno de los vértices del subcubo. Análogamente con los otros dos ejes, definiendo así el subcubo $2 \times 2 \times 2$. Se opera igual que anteriormente y el cubo resultante será propio o impropio en función del valor de la celda opuesta a t. Es decir, la única diferencia con el movimiento propio es el criterio para definir el subcubo.

Proceso de generación

En primer lugar, se comienza inicializando el cubo con un patrón fijo que satisface las restricciones explicadas. Llamamos a esta configuración *Initial cyclic cube*.

A continuación, se hace un ± 1-movimiento propio (porque la configuración inicial da lugar a un cubo de incidencia propio). Esto genera algunos cambios respecto al cubo original, obteniendo un cubo parecido al de la figura (b). Iteramos este proceso un número determinado de iteraciones *minIter*. Una vez se llega a el límite de iteraciones, simplemente se itera hasta obtener un cubo propio.

Llegados a este punto, tenemos un cubo como el de la figura (c). Simplemente queda pasarlo al equivalente cuadrado latino para tener lo que queríamos.

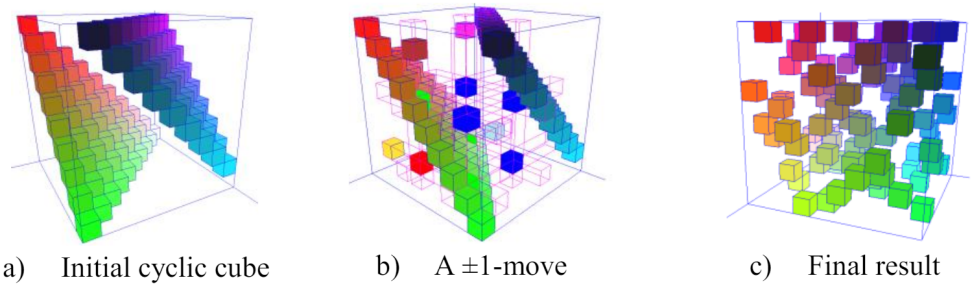


Figura 4.2: Proceso de generación de un cuadrado latino. Extraído de [5].

A continuación se muestra un pseudo-código de la generación de Kenkens:

$cube \leftarrow initialCyclicCube$
$proper \leftarrow$ `True`
$minIter \leftarrow n^3$
$iters \leftarrow 0$
while $iters < minIter$ & $!proper$ **do**
  $subcube = proper$? `subcubeProper()` : `subcubeImproper()`
  `compute()`
  $proper \leftarrow$ `updateProper()`
  $iters \leftarrow iters + 1$
end while
return `cubeToLatinSquare`($cube$)

Jacobson y Matthews demostraron que cualquier par de cubos (propios o impropios) están conectados por ±1-movimientos, en particular por menos de $2(n-1)^3$ movimientos (cota superior). Por tanto, el tiempo de generación de un cuadrado latino es polinómico respecto n, en particular, $O(n^3)$.

Implementación

Para implementarlo, nos hemos basado en el blog *Generating Random Latin Squares* [4], donde se ofrece una implementación del algoritmo en lenguaje *Go*. Esta ha sido traducida a Java para integrarlo en nuestro proyecto.

Esta implementación, en lugar de usar un `int[][][]` para representar un cubo de incidencia, usa tres `int[][]`. Estas corresponden a cada una de las tres vistas de los planos positivos xy, yz y xz. De esta manera, el cuadrado latino equivalente a al cubo de incidencia es directamente xy. Pese a no ser intuitivo, es la manera eficiente de hacerlo.

El objetivo de esta implementación es hacer que la búsqueda de 1s sea eficiente. Para ello, tenemos las siguientes variables (incluyendo las matrices anteriores):

- `xy[i][j] == k` $\Longrightarrow$ `cube(i,j,k) = 1`
- `yz[i][j] == i` $\Longrightarrow$ `cube(i,j,k) = 1`
- `xz[i][k] == j` $\Longrightarrow$ `cube(i,j,k) = 1`
- `proper` es un booleano que indica en todo momento si el cubo es propio
- `m` contiene las coordenadas de la celda que contiene un -1 cuando el cubo es impropio
- Las celdas (`m[0]`, `m[1]`, `mxy`), (`m[0]`, `mxz`, `m[2]`) y (`myz`, `m[1]`, `m[2]`) del cubo de incidencia también contienen un 1.

En definitiva, se implementa el algoritmo explicado usando estas estructuras y manteniendolas actualizadas para que sea eficiente.

4.2.2. Definir regiones

Dada la solución generada, queremos definir un conjunto de regiones que formen un Kenken válido cuya solución sea la generada. Este debería de cumplir todas las condiciones de Kenken a falta de ver que la solución sea única.

La idea es la siguiente: tenemos una matriz $n \times n$ de booleanos para marcar las casillas visitadas. Comenzamos por la casilla $(0, 0)$ y vamos visitando todas. Cada vez que encontremos una casilla no visitada, generamos el tamaño de la región que vamos a definir, incluimos esta casilla en esa región y nos movemos en una dirección aleatoria hacia otra casilla adyacente, con el objetivo de añadir casillas a la región hasta completarla.

Una vez tenemos un vector con las coordenadas de las casillas que conforman una región, podemos crear un vector con los operandos, es decir, los valores de la solución generada en las casillas de la región. A partir de aquí, queda escoger una operación para asignar a la región y calcular el resultado.

Elección del tamaño de las regiones

La elección del tamaño de las regiones es algo más delicado, pues afecta de manera determinante al rendimiento del generador. Cuantas más regiones tenga el Kenken (y por tanto, más pequeño sea su tamaño), más restricciones hay y, en consecuencia, más fácilmente el Kenken tendrá solución única. El caso extremo es cuando todas las regiones tienen tamaño 1: en ese caso la solución es siempre única. Sin embargo, tener una región de tamaño 10, por ejemplo, es menos restrictivo (habrá menos restricciones en total) y por tanto más dado a tener múltiples soluciones.

Teniendo esto en cuenta, se ha tratado de encontrar un equilibrio entre que el rendimiento sea bueno y que los Kenkens generados no sean aburridos o básicos. Para ello, se han

asignado diferentes probabilidades a distintos tamaños, entre tamaño 1 y 7. Las diferentes probabilidades han sido fruto de la experimentación, llegando a unos números que funcionan bastante bien.

Adicionalmente, no siempre se consideran estos tamaños: se limita el tamaño de las regiones a $\texttt{min}(3, n/2)$ para que no haya regiones demasiado grandes. Además, al inicio de la generación se descartan los tamaños que no encajen con ninguna de las operaciones escogidas. Por ejemplo, si se quiere generar un 9×9 solo con la resta (que solo admite dos operandos), inicialmente se considerarían los tamaños $1, 2, 3, 4 = 9/2$, pero los tamaños $3, 4$ se descartarían por no haber operaciones que los admitan. El 1 siempre es válido porque la identidad siempre forma parte de las operaciones admitidas. Dado que no siempre consideramos todos los tamaños, las probabilidades se escalan teniendo en cuenta las opciones que restan.

Comentar también que el tamaño 1 tiene generalmente probabilidad 0, pues no interesa tener demasiadas regiones de tamaño 1: el Kenken resultante sería fácil. Se podría poner una probabilidad baja, pero el sistema ya genera regiones de tamaño 1 de manera natural: cuando está intentando formar una región de tamaño k pero todas las casillas adyacentes ya forman parte de otras regiones, el tamaño escogido inicialmente se sobrescribe por el del número real de casillas que se han conseguido marcar.

Elección de las operaciones

El generador recibe un vector con los identificadores de las operaciones que el usuario quiere permitir en el Kenken generado. Esto quiere decir que no necesariamente el Kenken generado contará con todas las operaciones elegidas, simplemente no podrá usar aquellas no se hayan escogido.

Una vez tengamos una región definida con un determinado tamaño y unos operandos (valores de la solución en las casillas de la región), el sistema procede a escoger una operación aleatoriamente entre aquellas posibles. El programa intenta operar con el vector de operandos para obtener el resultado. Si lo consigue, esta será la operación escogida para la región. De lo contrario, se descarta y se coge otra operación. Esto puede pasar por varios motivos. El más común es que el tamaño de la región no es válido para la operación. Por ejemplo, la división y resta solo admiten tamaño de región 2. Sino, puede ser por otras restricciones que tengan las operaciones a la hora de operar. Por ejemplo, la división necesita que uno de los números sea divisible por el otro, para que la división de lugar a un número entero. Viendo este criterio, ya se observa que la división tendrá menos presencia en los Kenkens generados, pues por cualquiera de los 2 motivos esta será frecuentemente rechazada, pese a tener la misma probabilidad inicial que el resto de operaciones.

Adicionalmente, se puede dar el caso de que ninguna de las operaciones encaje con la región

que se ha definido. Esto puede pasar en la siguiente situación, por ejemplo: se define una región de tamaño 2 y las únicas operaciones son la identidad y la división. La identidad no encaja porque el tamaño tendría que ser 1. Si la división entre los dos números no da exacta, se descartará la división y ninguna de las operaciones habrá procedido con éxito. Cuando se produce este caso, se anula la iteración y se pasa a la siguiente. En general esta situación se produce poco, por lo que no nos importa prescindir de una iteración. En casos muy extremos, es muy posible que el sistema no sea capaz de generar un Kenken con los parámetros recibidos, por lo que esto acelerará el proceso de iteración y el impacto será positivo.

4.2.3. Comprobar la unicidad de la solución

Después de generar la solución y definir las regiones del Kenken, la única condición que falta verificar es que la solución generada es en efecto la única que satisface las restricciones definidas por las regiones. Para ello, usamos el método del Solver que comprueba exactamente esto: la unicidad de una solución. De esta manera, iteramos hasta encontrar una definición de regiones en la que la solución generada es la única. La estructura de la generación de Kenkens es algo parecido a lo siguiente:

$solution \leftarrow$ `LatinSquare(n)`
$uniqueSol \leftarrow$ `False`
$iters \leftarrow 0$
while $iters \leq maxIter$ & $!uniqueSol$ **do**
  $output \leftarrow$ `defineRegions(solution, operations)`
  $uniqueSol \leftarrow$ `solver.isUnique()`
  $iters \leftarrow iters + 1$
end while
return *output*

El bucle tiene un límite de iteraciones para contemplar el caso en el que la n y las operaciones no dan lugar a un Kenken, o este puede potencialmente tardar demasiado en generarse. Por ejemplo, si se pide un Kenken 9×9 con la división como única operación, el sistema no va a saber generar un Kenken con estas características. De esta manera, lo intentará hasta que llegue al máximo de iteraciones y retornará un `null`, indicando así que no se ha logrado generar el Kenken.

Además, también se ha implementando un *timeout* para las iteraciones, para acelerar aquellas iteraciones que tardan demasiado. Experimentando, hemos observado que hay iteraciones en las que el Solver tarda más de lo habitual en determinar la unicidad de la solución. Normalmente, cuando se quiere resolver un Kenken, no queda otra que esperar el tiempo que necesite, si el Kenken es difícil de resolver. Aquí, en cambio, no tenemos particular in-

terés en que el Kenken de la iteración tardía sea el que finalmente nos de solución única. Es más eficiente pasar a la siguiente iteración si está tardando demasiado. Para implementar esta funcionalidad, se ha definido la comprobación de la unicidad como una *task* y a continuación se manda a un *thread* a ejecutarla, poniendo un timer que haga que este acabe su ejecución pasado un tiempo determinado. El tiempo límite por iteración se puede ajustar. Nosotros, después de testearlo numerables veces, hemos optado por 3 niveles en función de la n. Concretamente:

$$\text{tiempo límite por iteración}(n) = \begin{cases} 10s & \text{si } n \in [3, 12] \\ 30s & \text{si } n \in [13, 15] \\ 60s & \text{si } n \geq 16 \end{cases}$$

4.2.4. Observaciones

Impacto de las operaciones en el rendimiento

A la hora de generar un Kenken obligatorio, hay 2 parámetros que escoge el usuario: la n (tamaño del tablero) y las operaciones. Mientras que el impacto del primero es evidente, cómo afectan las operaciones escogidas de cara a generar Kenkens no es tan directo. A continuación se explican algunas observaciones al respecto.

Recordamos que para que una iteración del generador sea exitosa, se tiene que comprobar que la solución ofrecida es la única que verifica las restricciones de las regiones definidas. Por tanto, cuánto más restrictivos sean las regiones definidas, más rápido irá el generador ya que alguna de las primeras iteraciones será exitosa. Por ejemplo, el caso más extremo es si se definen $n \times n$ regiones de 1×1, es decir, con la operación identidad. En este caso, en cada casilla estamos restringiendo que solo pueda ir un número, luego la solución es trivialmente única. Por tanto, la primera iteración sería exitosa y el proceso sería inmediato.

Por esta razón, podemos plantearnos cómo de restrictiva es cada operación. Sin entrar en detalle en cada una de ellas, destacaremos algunas. Excluyendo la identidad, la operación más restrictiva es el producto. Esto es intuitivo, ya que dado un número *result*, las combinaciones de números que multiplicados puedan dar lugar a este, son básicamente las maneras de combinar los números de su descomposición en factores primos, que en general no son demasiadas. Si lo comparamos con la suma, por ejemplo, es evidente que hay más maneras de sumar 3 números y que den 10, que con la multiplicación, ya que $10 = 2 \cdot 5$, por lo que las posibilidades son $1, 2, 5$ permutados de todas las formas posibles. De esta manera, generar Kenkens con solo la multiplicación (y la identidad, que es obligatoria) es la opción más rápida, con diferencia.

En el otro extremo se encuentra una de las operaciones añadidas por nosotros, el máximo. Ya se observa que decir que un conjunto de 5 casillas tiene un máximo de 7 no nos aporta

mucha información. En cada una de ellas, podría ir cualquier número entre el 1 y el 7. Por esto, si se intenta generar un Kenken con alta concentración de regiones con operación máximo, es posible que esta generación tarde mucho o que el sistema sobrepase el número límite de operaciones.

Time complexity

En resumen, la *pipeline* del generador consiste en generar un *Latin Square* ($\mathcal{O}(n^3)$) y a continuación iterar hasta un máximo de 200 iteraciones generando un potencial Kenken y comprobando si la solución es única. Sabiendo que la comprobación de la unicidad tiene un comportamiento exponencial, es claro que este coste domina al resto. De hecho, como el número de iteraciones es constante, podemos concluir que la complejidad del generador es la misma que la del Solver.

Sin embargo, en la práctica, el generador es lógicamente más lento que el Solver, pues en general tiene que hacer varias iteraciones antes de generar un Kenken con solución única.

4.3. Estructuras de datos a comentar

Region

Las estructura de datos más relevantes son:

- `Vector<Coordinates> cells`: usamos un vector ya que es más flexible que un *array* cuando se tenga que añadir elementos, como ocurre en la creadora. Aunque se podría cambiar fácilmente por un *array*, por razones de compatibilidad con otras clases que usan objetos *Region* mantenemos la estructura *vector*.

Kenken

Las estructura de datos más relevantes son:

- `Vector<Region> board`: representa la estructura del Kenken, según las regiones que lo componen. Usamos un vector por su flexibilidad, ya que esta estructura se utiliza en varios lugares del proyecto y, en algunos de estos lugares, es más conveniente usar un vector que un array.
- `int solution[][]`: aquí se guarda la solución correcta y oficial del Kenken. Usamos un *array* 2D de enteros, ya que es el tipo adecuado sabiendo el tamaño de la matriz y para poder acceder ágilmente a cada posición.

Game

Las estructura de datos más relevantes son:

- `Cell[][] board`, que representa el tablero de juego. Cada `Cell` consta de un `int selection`, que corresponde al número que el jugador elija poner en la celda, y un vector de booleanos `notes`, donde cada posición de 0 a $n-1$ se corresponde a un número de 1 a $n =$ `kenkenSize`, y representa si este está anotado (o no). Esta estructura es la más adecuada ya que está hecha *ad hoc* para las necesidades del juego. El *array* 2D, además, es la mejor opción para una matriz de tamaño fijo y para garantizar acceso fácil a cada posición.
- `Set<Coordinates> freePositions`: es un conjunto de coordenadas, donde se apuntan esas posiciones del tablero que el jugador aun no haya rellenado con una solución (no con notas). Esta estructura es necesaria para elegir donde dar una *hint* cuando el jugador la pida. Usamos un *set* ya que no queremos un orden particular, pero sobretodo porque el *set* no permite elementos repetidos: es decir, da más seguridad de que las posiciones que se obtienen no se hayan obtenido anteriormente, causando errores.

Ranking

La estructura de datos más importante es:

- `LinkedHashMap<String, Integer> ranking`: se almacena el ranking de usuarios con su nombre y su puntuación. Esta implementación de `Map` es conveniente porque se ordena por orden de inserción y no por orden en las llaves. De esta forma podemos ordenar el *ranking* por puntuación y no por nombre, con una función de ordenación propia (en caso de empate se ordena por orden lexicográfico del nombre de usuario).

Verificador

Las estructuras de datos más importantes son:

- `Vector<Vector<Integer>> integerFormat` donde se va guardando una traducción a enteros, quitando las pistas, del input. Consideramos que la estructura `Vector` era la adecuada ya que nos daba más facilidades a la hora de inserir elementos en cualquier posición del vector. Y además, la estructura es más flexible cuando necesitas un contenedor de un tamaño desconocido, en comparación con un array.
- `Vector<Vector<Integer>> hintsPositions`, los motivos son similares. Al no tener de antemano cuantas pistas vamos a tener en un input, un vector es una opción más cómoda que un array.

Operation

Hay una única estructura de datos a comentar: `Map<Integer, Operation> opMap`. Esta estructura permite un mapeo rápido y eficiente entre un identificador (ID) y la instancia correspondiente de la operación. Utilizar un `HashMap` permite acceder a las instancias de las operaciones en tiempo constante. Además, esta estructura asegura que cada clave sea única. En este caso, cada ID de operación se asocia a una única instancia de operación, evitando colisiones o duplicaciones que podrían ocurrir en otras estructuras.

Solver

Las estructura de datos más relevantes son:

- `Vector<Region> regions`: contiene las regiones que describen un potencial Kenken. Usamos un vector por su flexibilidad, ya que esta estructura se utiliza en varios lugares del proyecto y, en algunos de estos lugares, es más conveniente usar un vector que un array.

- `int[][] solution`: contiene la solución del Kenken. Se usa un array porque es la opción más eficiente y no se necesita nada más sofisticado.

GSolver

Las estructura de datos más relevantes son:

- `MPVariable[][][] boolVars` y `MPVariable[][] vars`: estas estructuras contienen las variables del sistema de inecuaciones. Se usan arrays porque es la opción más simple y es suficiente para satisfacer las necesidades que tenemos.
- `int[][] boardFactorization`: cada fila i es la factorización en números primos de $i+1$, donde $i \in \{0, \ldots, n-1\}$. Dado que la factorización de $1, \ldots, n$ se usa repetidamente, esta se precalcula y se guarda en esta variable. Esta estructura es la adecuada, pues tiene un tamaño fijo que sabemos de antemano y una vez inicializada, no se modifica: solo necesitamos que el acceso sea rápido.

Generator

Las estructura de datos más relevantes son:

- `double[] probabilities`: contiene las probabilidades que usamos para generar el tamaño de las regiones. Se usa un array porque es una opción simple y eficiente.
- `int[][] directions`: contiene las 4 direcciones en las que te puedes mover en el tablero. De igual manera, se usa un array por la simplicidad y eficiencia del mismo.

Además, pese a no ser atributos, destacamos las siguientes variables que se usan en algunas funciones de la clase:

- `Vector<Vector<Integer>> output`: contiene el potencial Kenken en formato estándar. Escogemos esta estructura porque es la que da el Verificador como output, y la que recibe el Traductor como input.
- `Boolean[][] vis`: matriz de booleanos donde se marcan las casillas ya visitadas mientras generamos las regiones. Se usa un array porque el tamaño es fijo, de manera que es una estructura eficiente y simple que cubre nuestras necesidades.

Capítulo 5

Operaciones añadidas

En este capítulo se explica la manera de añadir nuevas operaciones al juego. A modo de ejemplo, se describe cómo se han añadido las dos operaciones que hemos escogido añadir nosotros: la media aritmética (AM) y el máximo (max).

A parte de lo que se explica, también sería conveniente añadir tests que verifiquen la funcionalidad de la operación añadida. Es decir, tests que resuelvan kenkens que contengan la nueva operación.

5.1. Media aritmética (AM)

La primera operación que hemos escogido es la media aritmética, también llamada promedio o simplemente media. Esta se obtiene con la suma de un conjunto de valores dividida entre el número total de sumandos. Formalmente, dados n números $x_1, x_2, \ldots, x_n$, definimos la media aritmética como:

$$\overline{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

Para añadir la operación al sistema, en primer lugar tenemos que añadir la subclase de la nueva operación:

```
final class ArithmeticMean extends GOperation {
    @Override
    public int operate(Vector<Integer> operands) {
        if (!correctNumOperands(operands.size())) {
            throw new IllegalArgumentException(
                "Number of operands not correct");
        }
        return operands.stream().reduce(0, Integer::sum)/operands.size();
    }

```

```
11      @Override
12      public void linearize(GSolver s, Region r) {
13          MPConstraint sumConstraint1 = s.solver.makeConstraint(
14              -s.infinity, r.getTarget()*r.getCells().size() , "");
15          MPConstraint sumConstraint2 = s.solver.makeConstraint(
16              r.getTarget()*r.getCells().size(), s.infinity,  "");
17
18          for (Coordinates c : r.getCells()) {
19              sumConstraint1.setCoefficient(s.vars[c.getX()][c.getY()], 1);
20              sumConstraint2.setCoefficient(s.vars[c.getX()][c.getY()], 1);
21          }
22      }
23  }
```

Listing 5.1: Nueva subclase ArithmeticMean

Los métodos que tenemos que añadir son los siguientes:

- `getMinOperands()`: retorna el número mínimo de operandos que admite la operación (por defecto 2 si no se implementa).
- `getMaxOperands()`: retorna el número máximo de operandos que admite la operación (por defecto `Integer.MAX_VALUE` si no se implementa).
- `operate(Vector<Integer> operands)`: retorna el resultado de aplicar la operación a los elementos de un vector.
- `linearize(GSolver s, Region r)`: define las restricciones que definen la operación sobre las casillas de la región.

En este caso, la media aritmética es un derivado de las operaciones clásicas ya implementadas, por lo que es fácil implementar la linealización. En particular, podemos aprovechar las restricciones de la suma, puesto que

$$\frac{x_1 + x_2 + \cdots + x_n}{n} = \text{target} \iff x_1 + x_2 + \cdots + x_n = n \cdot \text{target}$$

Así, usamos el código de la suma, multiplicando *target* por el número de casillas de la región. Lo único que hemos tenido que añadir es `r.getCells().size()` en las líneas 14 y 16, multiplicando `r.getTarget()`.

Finalmente, añadimos la nueva clase, junto con el `operationId` que le queremos asignar, en el *map* de la clase Operation.java. En este caso, así queda después de añadir la media aritmética con Id = 5 a las operaciones estándar.

```
1  static {
2      opMap.put(0, new Identity());
3      opMap.put(1, new Addition());
```

```
    opMap.put(2, new Subtraction());
    opMap.put(3, new Product());
    opMap.put(4, new Division());
    opMap.put(5, new ArithmeticMean());
}
```

Listing 5.2: Map de la clase Operation.java

5.2. Máximo (max)

Para la segunda operación, hemos optado por el máximo. Dados n números $x_1, x_2, \ldots, x_n$, definimos el máximo como el mayor de los x_i. Formalmente, las restricciones que definen esta operación son las siguientes:

- $\max \geq x_i \quad \forall i \in \{1, \ldots, n\}$
- $\exists i \in \{1, \ldots, n\}$ tq $\max = x_i$

Usamos esto para hacer la función de linealización:

```
@Override
public void linearize(GSolver s, Region r) {
    // the value of each cell <= target
    for (Coordinates c : r.getCells()) {
        MPConstraint valueSmallerThanTarget = s.solver.makeConstraint(
            -s.infinity, r.getTarget() , "");
        valueSmallerThanTarget.setCoefficient(
            s.vars[c.getX()][c.getY()], 1);
    }

    // there exists a cell whose value is target
    MPConstraint existsValueTarget = s.solver.makeConstraint(
        1.0, s.infinity,  "");
    for (Coordinates c : r.getCells()) {
        existsValueTarget.setCoefficient(
            s.boolVars[c.getX()][c.getY()][r.getTarget()-1], 1);
    }
}
```

Listing 5.3: Función de linealización de Max

El resto de cambios son análogos a los mostrados para la media aritmética.

Capítulo 6

Juegos de prueba

6.1. Juegos de prueba con persistencia

Pasos para usar un juego de pruebas (partiendo de la aplicación cerrada):

- Entrar al directorio `EXE`
- Ejecutar `./juegoDePrueba.bat` *nombre*
- La aplicación se abre con el juego de pruebas cargado. Ya se puede proceder a seguir los pasos del juego de pruebas correspondiente.

Nota: Para agilizar los juegos de prueba, en todos ellos se ha añadido al sistema un usuario especial:

- username: `root`
- password: `root`

De esta manera, se puede entrar rápidamente al juego y ver el estado del sistema, aunque después se quiera entrar con otro perfil.

49UsersRanking

Objetivo: comprobar que cuando hay 49 usuarios en el sistema y se añade uno nuevo, este aparece en el ranking.

Descripción de uso:

- Si entramos con el usuario `root`, podemos consultar el ranking y ver que se muestran los 49 usuarios y que todos tienen más de 0 puntos.
- Registramos un nuevo usuario. Ejemplo de nuevo usuario que funciona:

username: Bernat
contraseña: P1234567890'

- Consultar el ranking
- Comprobar que, en efecto, el nuevo usuario aparece en el ranking

Justificación: el ranking muestra los 50 primeros usuarios (ordenados por puntos y lexicográficamente). Este juego de pruebas trata de comprobar que si hay menos de 51 usuarios en el sistema, todos aparecen en el ranking, independientemente de la puntuación.

50UsersRanking

Objetivo: comprobar que cuando hay 50 usuarios (con más de 0 puntos) en el sistema y se añade uno nuevo, este no aparece en el ranking.

Descripción de uso:

- Si entramos con el usuario `root`, podemos consultar el ranking y ver que se muestran los 50 usuarios y que todos tienen más de 0 puntos.
- Registramos un nuevo usuario. Ejemplo de nuevo usuario que funciona:
  username: Ian
  contraseña: P1234567890'
- Consultar el ranking
- Comprobar que, en efecto, el nuevo usuario no aparece en el ranking

Justificación: el ranking muestra los 50 primeros usuarios (ordenados por puntos y lexicográficamente). Este juego de pruebas trata de comprobar que si hay 51 usuarios en el sistema, aquel con menor puntuación no aparece en el ranking.

51UsersRanking

Objetivo: comprobar que cuando hay 51 usuarios en el sistema y se borra 1, aquel que no aparecía ahora sí aparece. Alternativamente, si este último puntúa lo suficiente para sobrepasar al número 50, este aparece en el ranking y el que estaba en posición 50 desaparece.

Descripción de uso:

- Si entramos con el usuario `root`, podemos consultar el ranking y ver que se muestran 50 usuarios (todos con más de 0 puntos) de los 51 que hay en el sistema.
- Borramos un usuario.
  - Profile → Log out

    - Log in
    - username: Able
      contraseña: m2ZkI*:-
    - Profile → Eliminate profile

- Iniciamos sesión con el usuario `root`.
- Consultar el ranking
- Comprobar que, en efecto, hay 50 usuarios y se muestran todos en el ranking.

Justificación: el ranking muestra los 50 primeros usuarios (ordenados por puntos y lexicográficamente). Este juego de pruebas trata de comprobar que si hay 51 usuarios en el sistema, el sistema responde bien cuando el último posicionado puntúa, o cuando se borra un usuario del top 50.

RankingSorted

Objetivo: comprobar que el ranking ordena a los usuarios lexicográficamente en caso de empate.

Descripción de uso:

- Si entramos con el usuario `root`, podemos consultar el ranking y ver que se muestran todos los usuarios y están ordenados correctamente.
- Registramos un nuevo usuario. Ejemplo de nuevo usuario que funciona:
  username: Dante
  contraseña: P1234567890'
- Consultar el ranking
- Comprobar que, en efecto, el nuevo usuario aparece en el ranking y aparece en la posición correcta en función de su nombre (en el caso del ejemplo, junto al usuario cuyo nombre empieza por D)

Justificación: el ranking muestra los 50 primeros usuarios (ordenados por puntos y lexicográficamente en caso de empate). Este juego de pruebas trata de comprobar que si hay 26 usuarios con 0 puntos, cada uno de ellos con un username que empieza por una letra diferente, el ranking los ordena lexicográficamente. Además, si añadimos uno, observamos que este se coloca en el ranking en la posición que le corresponda según su username.

Just5Users

Objetivo: comprobar que el sistema no deja crear una cuenta con un nombre que ya existe.

Descripción de uso:

- Entrar en register
- Introducir como username uno de los siguientes: Alex, Federica, Carlos, Jan. Todos ellos están en el sistema.
- Introducir una contraseña válida que cumpla los requisitos, por ejemplo: P1234567890'
- Comprobar que, en efecto, el sistema no permite registrar el nuevo usuario

Justificación: el username de un usuario ha de ser único en el sistema. Por tanto, no se ha de permitir crear usuarios con usernames repetidos.

ContinueGame

Objetivo: comprobar que cuando tenemos una partida a medias se puede continuar.

Descripción de uso: Tenemos dos formas de comprobar esta funcionalidad. Hemos puesto a disposición del usuario dos kenkens para probar ambas formas. La primera:

- Hacemos login con un usuario. Por ejemplo, root.
- Pulsar el botón Play Game
- Pulsar uno de los kenken
- Pulsar Continue Game en el pop up
- Comprobar que se abre un Kenken

La segunda:

- Hacemos login con un usuario. Por ejemplo, root.
- Pulsar el botón Continue Game
- Pulsar una instancia de partida empezada
- Comprobar que se abre un Kenken

Justificación: Este juego de pruebas trata de comprobar que si hay una partida empezada en persistencia se puede continuar desde cualquiera de las dos formas que ofrecemos.

ResetGame

Objetivo: comprobar que cuando tenemos una partida a medias se puede reiniciar. Hemos puesto a disposición del usuario dos kenkens para probar ambas formas.

Descripción de uso: Tenemos dos formas de comprobar esta funcionalidad. La primera:

- Hacemos login con un usuario. Por ejemplo, root.
- Pulsar el botón Play Game
- Pulsar uno de los kenken
- Pulsar Delete game and start a new one en el pop up
- Comprobar que el Kenken está reiniciado

La segunda:

- Hacemos login con un usuario. Por ejemplo, root.
- Pulsar el botón Continue Game
- Pulsar una instancia de partida empezada
- Pulsar Restart en la partida que se abre
- Pulsar Restart en el pop up
- Comprobar que se ha reiniciado el Kenken

Justificación: Este juego de pruebas trata de comprobar que si hay una partida empezada en persistencia se puede reiniciar desde cualquiera de las dos formas que ofrecemos.

EndGameNoHints

Objetivo: comprobar que podemos acabar una partida correctamente con sin usar ninguna pista.

Descripción de uso:

- Hacemos login con un usuario. Por ejemplo, root.
- Pulsar el botón Continue Game
- Pulsar una instancia de partida empezada
- Completar la casilla restante correctamente

Justificación: Este juego de pruebas trata de comprobar si las mecánicas implicadas en el final de la partida funcionan correctamente. La partida debería terminar correctamente

y anunciar los resultados pertinentes. Al salir, si consultamos las estadísticas del jugador estas (excepto juegos perdidos) deberían de ser diferentes de cero y la puntuación del jugador debería de coincidir con la anunciada al final de partida. Dependiendo de si hemos completado la última casilla con una pista o no, la estadística de juegos ganados sin pistas debería diferir. La puntuación obtenida en ambas situaciones tambien será diferente.

EndGameUsingHints

Objetivo: comprobar que podemos acabar una partida correctamente usando pistas.

Descripción de uso:

- Hacemos login con un usuario. Por ejemplo, root.
- Pulsar el botón Continue Game
- Pulsar una instancia de partida empezada
- Completar la casilla restante correctamente

Justificación: Este juego de pruebas trata de comprobar si las mecánicas implicadas en el final de la partida funcionan correctamente al utilizar pistas. La partida debería terminar correctamente y anunciar los resultados pertinentes. Al salir, si consultamos las estadísticas del jugador estas (excepto juegos perdidos y juegos ganados sin usar pistas) deberían de ser diferentes de cero y la puntuación del jugador debería de coincidir con la anunciada al final de partida. Dependiendo de si hemos completado la última casilla con una pista o no, la puntuación obtenido por completar la partida (si no hemos superado los tres minutos de tiempo) debería diferir de 10 puntos.

EndGameWithDifferentTimes

Objetivo: comprobar que al acabar una partida con tiempos diferentes, esto repercute en la puntuación obtenida.

Descripción de uso:

- Hacemos login con un usuario. Por ejemplo, root.
- Pulsar el botón Continue Game
- Pulsar una instancia de partida empezada.
- Completar la casilla restante sin fallos y sin pistas para no repercutir en la puntuación.

Justificación: Este juego de pruebas trata de comprobar si las mecánicas implicadas en el final de la partida respecto al tiempo funcionan correctamente. La partida debería terminar

correctamente y anunciar los resultados pertinentes. En cada una de las 5 partidas predefinidas en el juego de pruebas deberíamos obtener una puntuación diferente, tras completarlos todos y comprobar las diferencias en las puntuaciones sabremos que esta funcionalidad funciona correctamente.

Empty

Objetivo: comprobar que al jugar el juego sin nada en persistencia sigue funcionando.

Descripción de uso:

- Libre. A elección del usuario.

Justificación: Este juego de pruebas trata de comprobar que si al usar la aplicación con normalidad desde un estado en el que no existe persistencia todo debería de funcionar con normalidad.

Default

Objetivo: comprobar que al jugar el juego desde un estado de persistencia genérico sigue funcionando.

Descripción de uso:

- Libre. A elección del usuario.

Justificación: Este juego de pruebas trata de comprobar que si al usar la aplicación con normalidad desde un estado genérico de persistencia todo debería de funcionar con normalidad.

6.2. Juegos de prueba sin persistencia

Disponemos de dos tipos de juegos de pruebas que no involucran persistencia. El primero son sencillamente una serie de contraseñas que no cumplen con los criterios que imponemos al registrar un nuevo usuario. El segundo es una serie de Kenkens en formato estándar que nos permitirán probar la funcionalidad de importar Kenken con distintos formatos correctos y muchos otros incorrectos.

En primer lugar, enumeramos las contraseñas que sí son válidas y la justificación correspondiente:

- Password: Passw0rd!
  Explicación: Cumple con todas las condiciones. Importante para validar que el sistema acepte una contraseña que cumpla con todos los requisitos.

- Password: P@ssw0rd
  Explicación: Exactamente 8 caracteres. Verifica que el sistema acepte contraseñas de exactamente la longitud mínima que cumplen con todos los demás requisitos.
- Password: Pa$$w0rd
  Explicación: Cumple con todas las condiciones con múltiples caracteres especiales. Asegura que el sistema pueda manejar contraseñas con más de un carácter especial.
- Password: Pa55w@rd1
  Explicación: Cumple con todas las condiciones con un carácter numérico en el medio. Asegura que el sistema pueda manejar caracteres numéricos no solo al final de la contraseña.
- Password: Pa$$w0rd!
  Explicación: Cumple con todas las condiciones con caracteres especiales complejos. Verifica que el sistema pueda manejar contraseñas con caracteres especiales complejos.
- Password: 1234Abc@
  Explicación: Todas las condiciones cumplidas en un orden diferente. Asegura que el sistema acepte contraseñas válidas sin importar el orden de los caracteres que cumplen con las condiciones.
- Password: NOLOWERCASE7!
  Explicación: Cumple todas las condiciones y no tiene letra minúscula. Confirma que el sistema no requiere letras minúsculas.

Ahora, aquellas contraseñas que no son válidas y la justificación correspondiente:

- Password: short7!
  Explicación: Menos de 8 caracteres. Verifica que el sistema rechace contraseñas que no cumplen con el requisito de longitud mínima.
- Password: nouppercase7!
  Explicación: No tiene letra mayúscula. Asegura que el sistema rechace contraseñas sin una letra mayúscula.
- Password: NoNumber!
  Explicación: No tiene carácter numérico. Verifica que el sistema rechace contraseñas que carecen de un número.
- Password: NoSpecial7
  Explicación: No tiene carácter especial. Asegura que el sistema rechace contraseñas sin un carácter especial.
- Password: Password123

Explicación: No tiene carácter especial. Confirma que el sistema requiera al menos un carácter especial.

- Password: 12345678
  Explicación: No tiene letra mayúscula, no tiene carácter especial. Verifica que el sistema rechace contraseñas sin una letra mayúscula y sin un carácter especial.

- Password: abcdefgh
  Explicación: No tiene letra mayúscula, no tiene número, no tiene carácter especial. Confirma que el sistema rechace contraseñas que no cumplen con ninguno de los requisitos adicionales más allá de la longitud.

- Password: ABCDEFGH
  Explicación: No tiene letra minúscula, no tiene número, no tiene carácter especial. Asegura que el sistema rechace contraseñas que no cumplen con ninguno de los requisitos adicionales más allá de la longitud.

- Password: 1234567!
  Explicación: No tiene letra mayúscula. Valida que el sistema haga cumplir el requisito de una letra mayúscula.

Pasamos a enumerar los Kenkens incorrectos y la funcionalidad que desean comprobar (utilizaremos la notación N y R que figura en el formato estándar):

- `KenkenWrong3x3_1DifferentFormat_1.txt`: Tras importar el Kenken correcto *Kenken3x3_1.txt*, este Kenken contiene una permutación de sus regiones para comprobar que se detecta de forma correcta que se trata del mismo Kenken.

- `KenkenWrong3x3_1DifferentFormat_2.txt`: Tras importar el Kenken correcto *Kenken3x3_1.txt*, este Kenken contiene una permutación de las celdas de algunas de sus regiones para comprobar que se detecta de forma correcta que se trata del mismo Kenken.

- `KenkenMultipleSolution.txt`: El Kenken tiene múltiples soluciones válidas, lo cual no es permitido ya que debe tener una única solución.

- `KenkenNoSolution.txt`: El Kenken no tiene ninguna solución posible.

- `KenkenWrongCoordinateOutOfBound.txt`: Hay una coordenada fuera del rango permitido (entre 1 y N).

- `KenkenWrongCoordinateRepeated.txt`: Una coordenada está repetida en la definición de las regiones.

- `KenkenWrongHeader1.txt`: El parámetro R no aparece en el encabezado.

- `KenkenWrongHeader2.txt`: El valor de N en el encabezado no es un número entero positivo.
- `KenkenWrongHeader3.txt`: El valor de R en el encabezado no es un número entero positivo.
- `KenkenWrongHintFormat.txt`: El formato de una pista no sigue el estándar especificado.
- `KenkenWrongMisplacedHint.txt`: Una pista está ubicada en una posición incorrecta en el formato estándar.
- `KenkenWrongNotNumerical.txt`: Uno de los valores que deberían ser enteros no es un número.
- `KenkenWrongNumberOfR.txt`: El número de regiones no coincide con el valor especificado de R en el encabezado.
- `KenkenWrongNumberOfN.txt`: La dimensión del kenken no coincide con el valor especificado de N.
- `KenkenWrongOp.txt`: La operación especificada no está dentro del rango permitido (0 a 6).

Los kenkens correctos comprueban varias dimensiones para asegurarse de que se procesan correctamente. En particular, tenemos los siguientes:

- `Kenken3x3_1.txt` y `Kenken3x3_2.txt`: Contienen kenkens 3x3.
- `Kenken5x5.txt` Contiene un kenken 5x5.
- `Kenken6x6.txt` Contiene un kenken 6x6 y comprueba adicionalmente a añadir pistas.
- `Kenken9x9_1.txt` y `Kenken9x9_2.txt` Contienen kenkens 9x9.

Bibliografía

[1] Melkonian, V. (2016) An Integer Programming Model for the KenKen Problem. American Journal of Operations Research, 6, 213-225. http://dx.doi.org/10.4236/ajor.2016.63022

[2] Google OR-Tools website. https://developers.google.com/optimization

[3] Jacobson M., Matthews P.: Generating uniformly distributed random Latin squares. J.Combin. Des., 4(6):405–437 (1996). https://doi.org/10.1002/(SICI)1520-6610(1996)4:6%3C405::AID-JCD3%3E3.0.CO;2-J

[4] Paul Hankin (2019) Generating random latin squares. https://blog.paulhankin.net/latinsquares/

[5] I. Gallego Sagastume, "Generation of Random Latin Squares Step By Step and Graphically". https://sedici.unlp.edu.ar/bitstream/handle/10915/42155/Documento_completo.pdf?sequence=1

Anexos

Anexo A

Verificador, en detalle

El objetivo de esta clase es verificar si dado una entrada del formato estándar esta cumple las condiciones estructurales necesarias para poder ser enviada al Solver que comprobará si la entrada tiene solución única i así poder instanciar un objeto KenKen.

La clase gira alrededor de un método público llamado `verify` que recibe por entrada un `Vector<String>`, representando la entrada a evaluar y devuelve un `boolean` que codifica si la entrada tiene formato correcto. Esta función verifica las siguientes características del Input:

- Primero se divide el vector en un `Vector<Vector<String>>` para poder tratar los elementos 1 a 1.
- Comprobar que la cabecera de la entrada sea de tamaño 2, que el número de regiones sea menor o igual al número de casillas al cuadrado y que el tamaño sea mayor que 1 y que el número de regiones sea mayor o igual a 1.
- Comprobar que en la entrada se codifican R regiones (i.e. el `Vector<Vector<String>>` está compuesto de R+1 (para contar la cabecera) `Vector<String>`.
- Se traduce la cabecera a un `Vector<Integer>` y se añade a un `Vector<<Vector>Integer>` llamado *integerFormat*, que se va a ir completando a lo largo de la verificación donde se va a guardar una traducción a enteros del formato estándar con la pistas eliminadas. La creación de esta traducción es necesaria para la clase que traduce el `Vector<<Vector>Integer>` a un `Vector<Region>` que se pueda mandar al *Solver* para que busque si tiene solución.
- Para cada región se define un `Vector<Integer>` *region* donde se va almacenar la región traducida a enteros y se comprueba:
    - Que los `String` que representan los números de operación, resultado y número

de casillas sean números y traducirlos a enteros. Comprobar que el número de operación corresponda a una operación válida.

- Que el número de casillas de la región se adecue a sus requisitos (por ejemplo, que en una resta solo haya dos casillas).
- Comprobar que el número de casillas que se codifica en la entrada corresponde con el número de casillas introducidas.
- En caso que se verifiquen las tres condiciones se añaden los tres valores a *region*.
- Para cada `String` del vector comprobar:
  - Si está en una posición dónde puede ir una pista (después de una coordenada):
    - Comprobar si el primer carácter del `String` es '[', es decir que estamos empezando a leer una pista.
    - En caso afirmativo, leemos el sub-`String` entre la segunda y penúltima posición (incluidas ambas) y comprobamos que es un número y lo traducimos.
    - Leemos el último carácter del `String` y comprobamos si es un ']'. En caso afirmativo concluimos que hemos leído una pista de forma satisfactoria. Se procede a marcar en una matriz de pistas (inicializada con -1), que en esa posición tenemos la pista encontrada, se aumenta el iterador del bucle en 1 y se pone a `false` el `boolean` *isParenthesisReady*, que nos indica que ahora hace falta leer una coordenada antes de leer otra pista.
    - En caso de que no se den cualquiera de los dos últimos puntos se devuelve falso.
  - En caso contrario:
    - Se leen el `String` sobre el que estamos y el siguiente y se comprueba que son números enteros y que exista la siguiente posición en el vector. En caso afirmativo estos dos valores pasan a ser los candidatos de ser las coordenadas de la casilla de la región. Se actualiza el `boolean` *isParenthesisReady* a `true` para indicar que ahora podríamos leer una pista. Se aumenta el iterador del bucle en dos (ya que hemos leídos dos números).
    - En caso de no cumplirse las dos condiciones anteriores devolver `false`.
  - En caso de que estemos en una posición donde no puede ir una pista repetimos el procedimiento explicado para cuando leemos las coordenadas en el caso

anterior.

- En caso de que hayamos leído unas coordenadas (*isParenthesisReady* `true`) se compruebas:
  - Que las coordenadas leídas estén dentro del tablero.
  - Que la casilla definida por las coordenadas no pertenezca a otra región.
  - Se añaden las coordenadas a *region*
- Una vez operados todos los `String` de la región. Se comprueba que la región definida tenga una componente conexa (explicación en elaborada en el siguiente párrafo).
- Se comprueba que tenga el tamaño anunciado por el número de casillas leído al inicio de la región.
- Se añade *región* a *IntegerFormat*

- Después de tratar todas las regiones se comprueba que todas las casillas del tablero pertenecen a alguna región.

En caso de verificar todas estas condiciones el método devuelve true y en caso contrario false. Vamos a detallar el algoritmo seguido para determinar que las regiones solo tienen una componente conexa pues es el algoritmo más interesante. Para empezar, nosotros entendemos una región es conexa si todas sus casillas se puede visitar haciendo solo movimientos en vertical y horizontal dentro de la región. El algoritmo básicamente recibe una matriz de booleanos con todas las entradas a false excepto las casillas de la región y una `Set<int[]>` con las coordenadas de las casillas de la región. Entonces tras comprobar que la lista de coordenadas no es vacía se realiza una DFS con el objetivo de recorrer todas la región marcada como `true` en la matriz que recibe como entrada. Guardamos las posiciones que hemos podido visitar empezando por la primera (en orden de aparición en la entrada del formato estándar), comprobamos que están todas las del Set de entrada y en caso afirmativo se devuelve un `true`.